

Лекция 11.

Графовые нейросетевые модели.



Миша Печатов

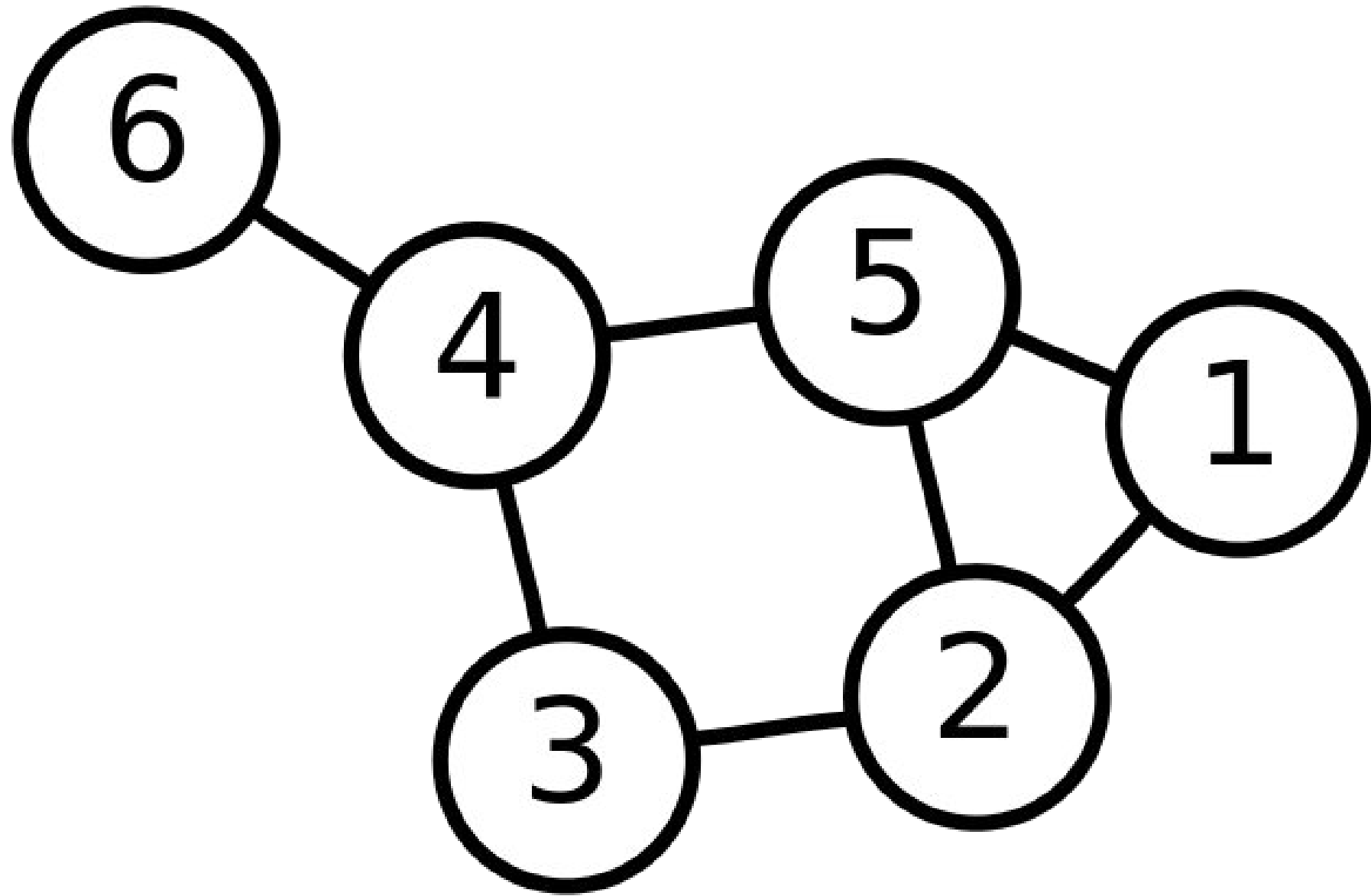
План

- ➔ Графы
- ➔ Примеры задач на графах
- ➔ Графовые свертки
- ➔ Графовые нейросети



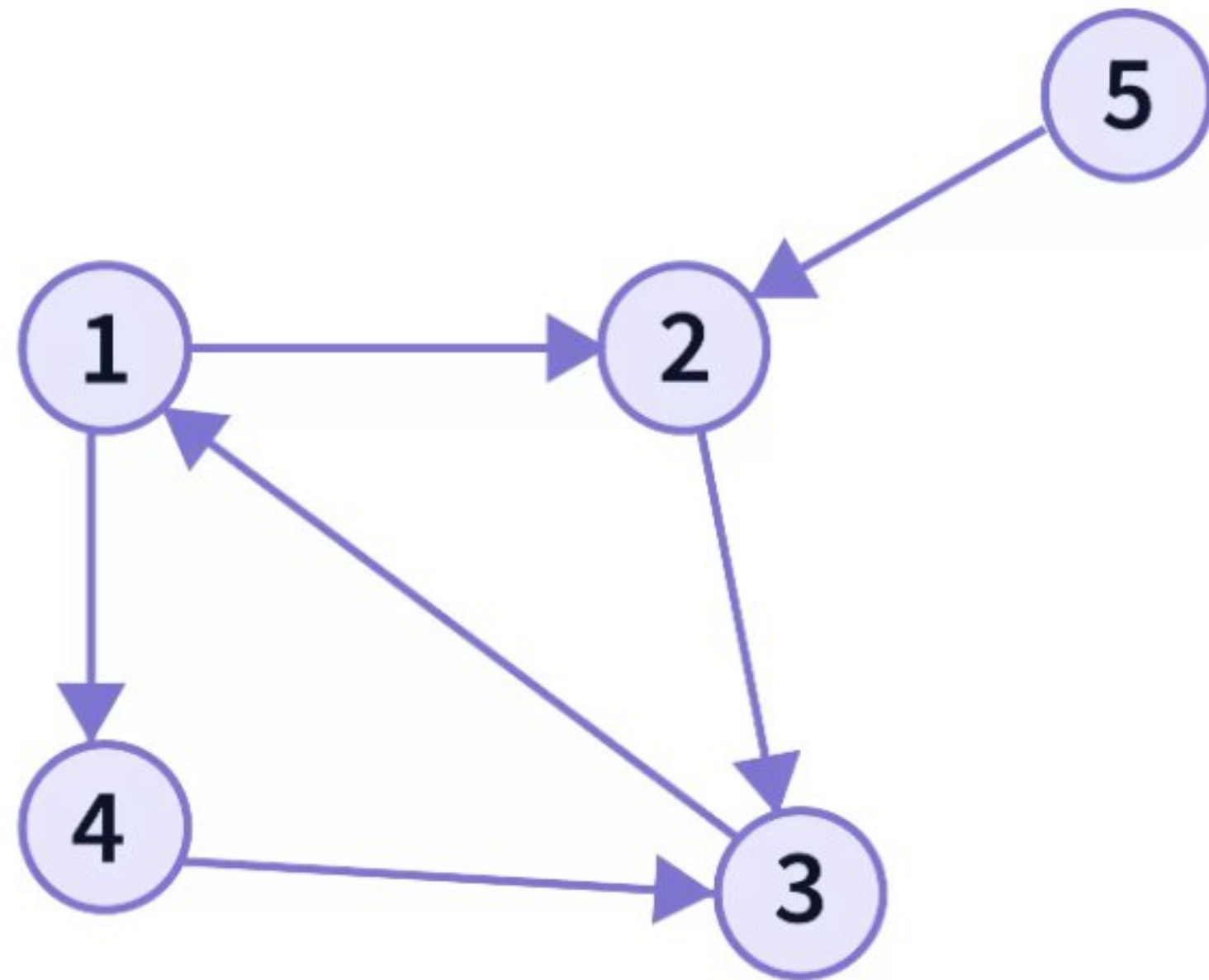
Графы

Графы

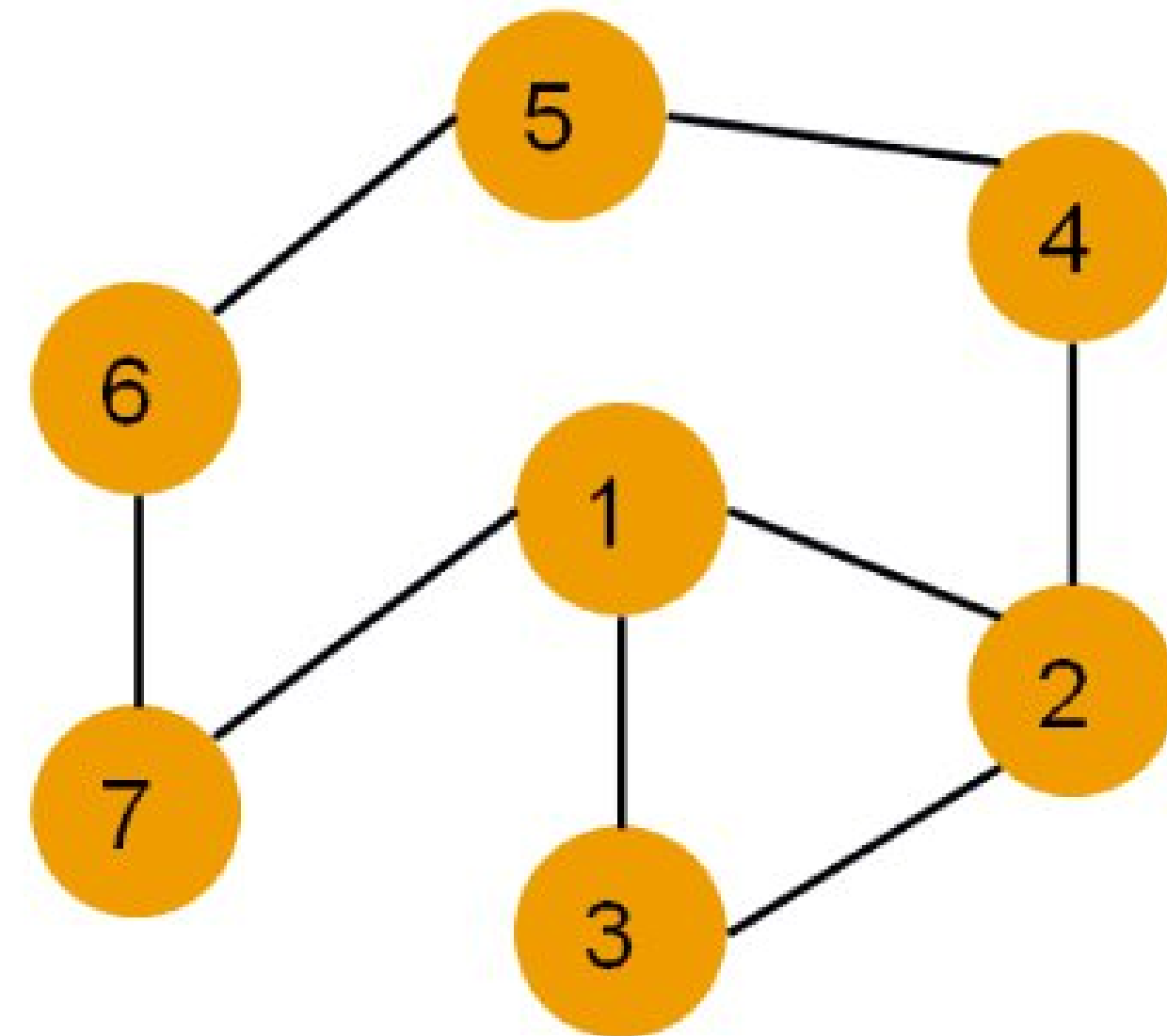


Граф $G(V, E)$ - объект, задающийся двумя множествами **вершин** и **ребер**

Графы

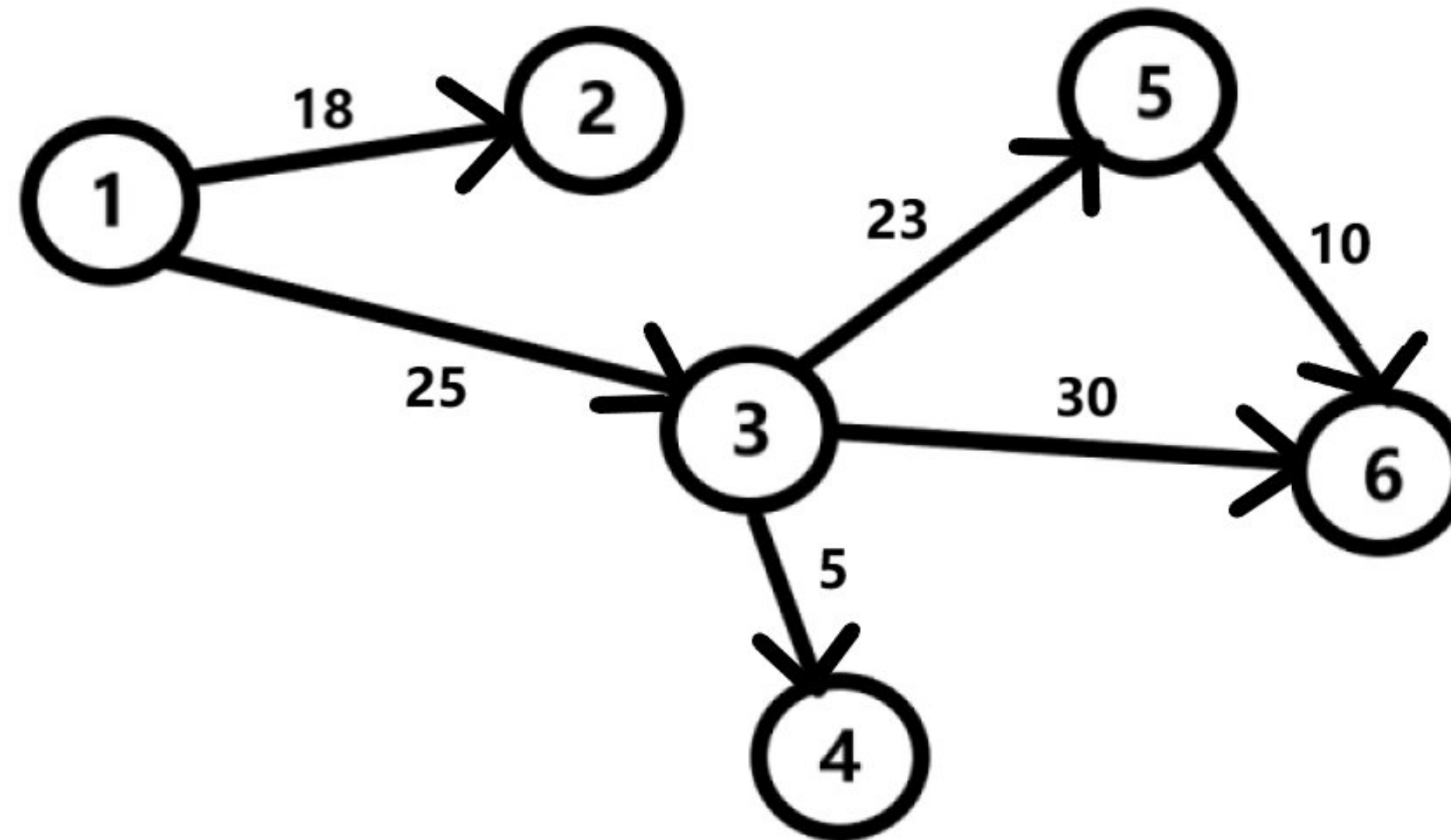


Ориентированный



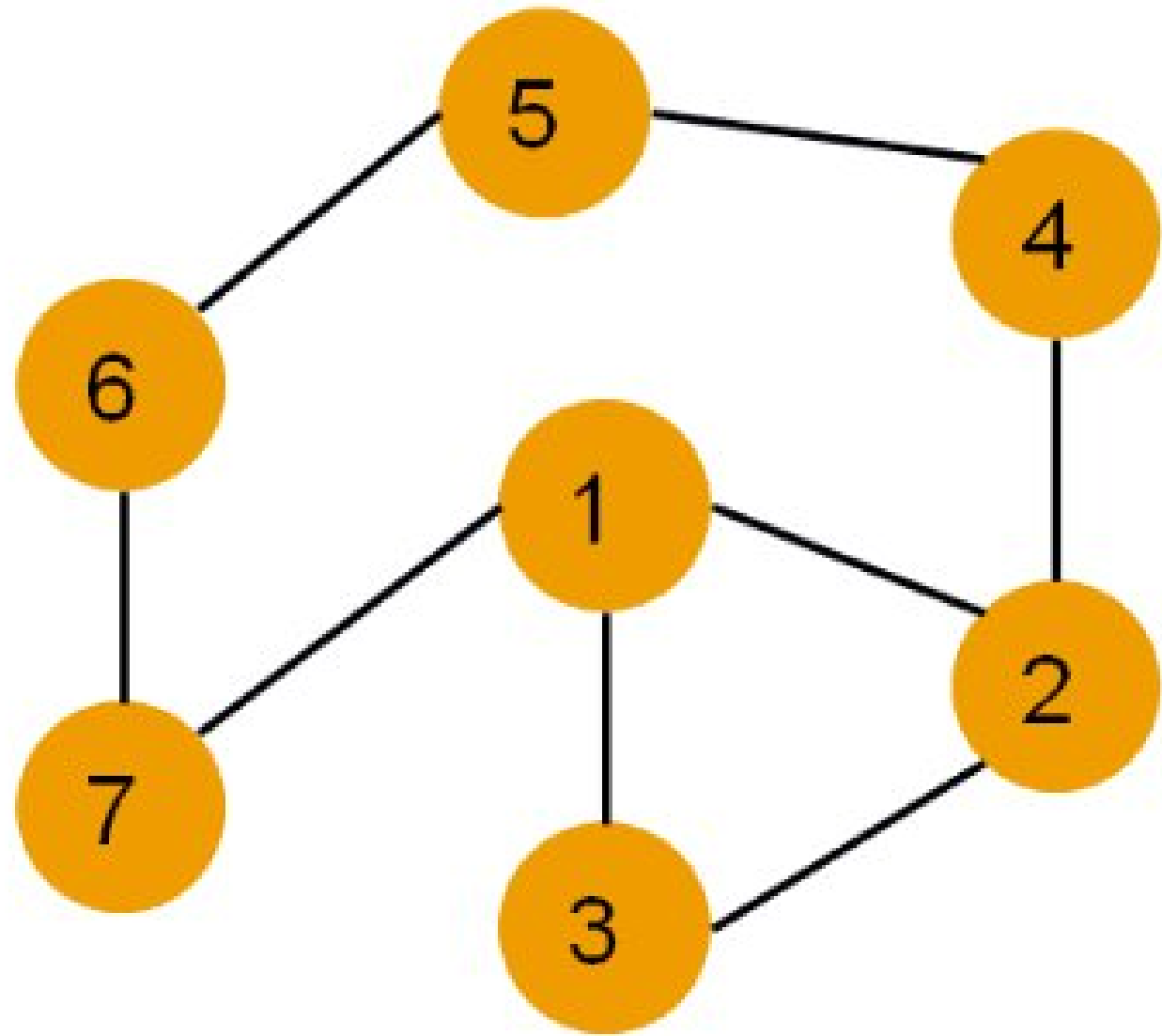
Неориентированный

Графы

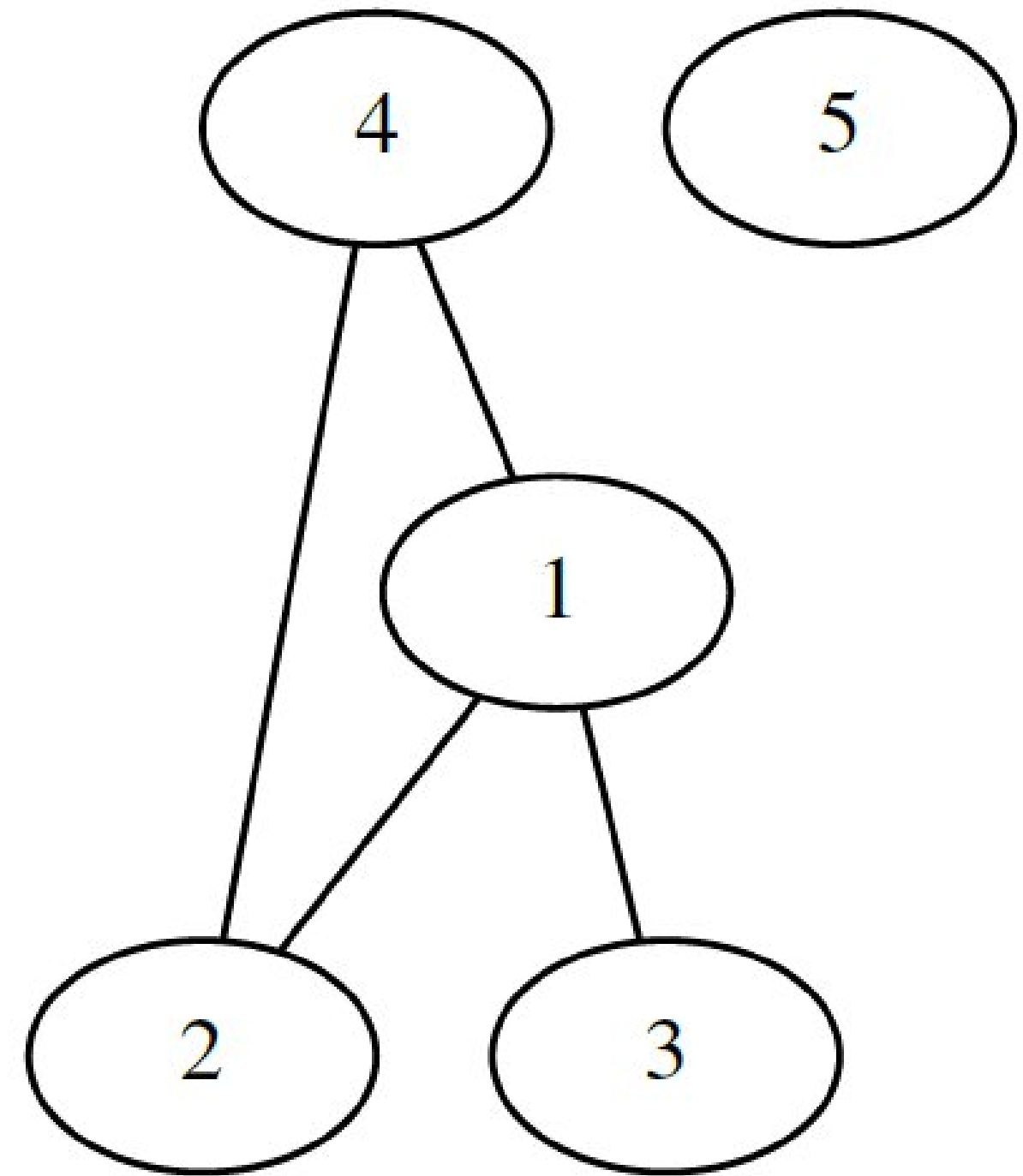


Взвешанный

Графы

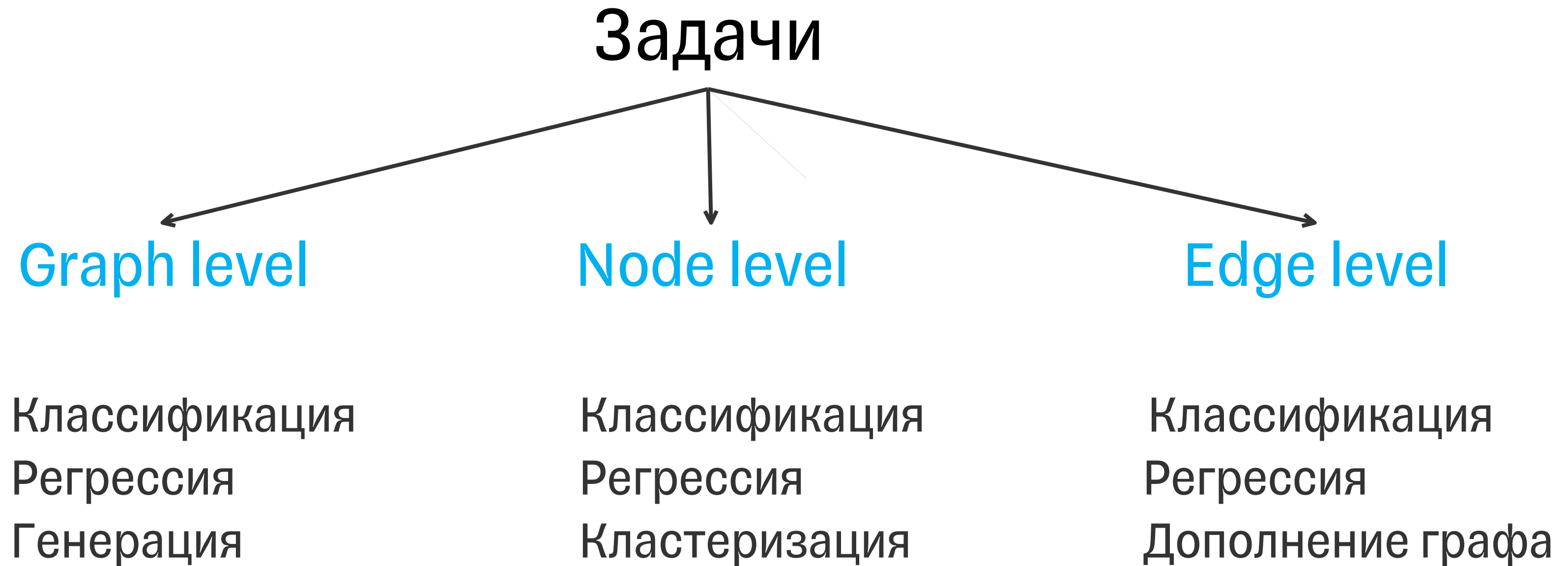


СВЯЗНЫЙ

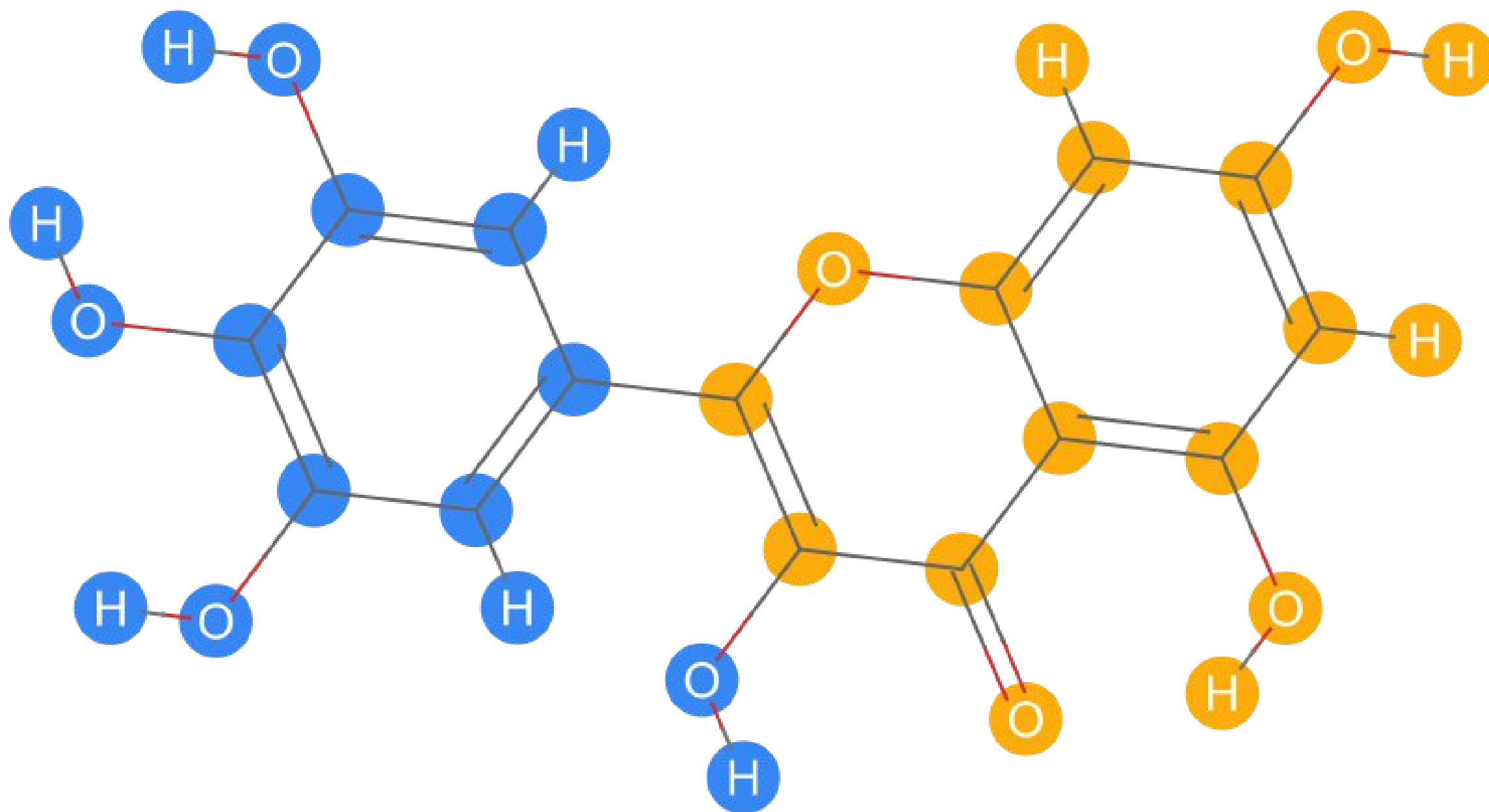


Несвязный

Задачи на графах



Graph level



Классификация: является антибактериальным/токсичным

Регрессия: Характеристика липофильности молекул

Генерация: Сгенерировать молекулу обладающую нужными свойствами

Node level

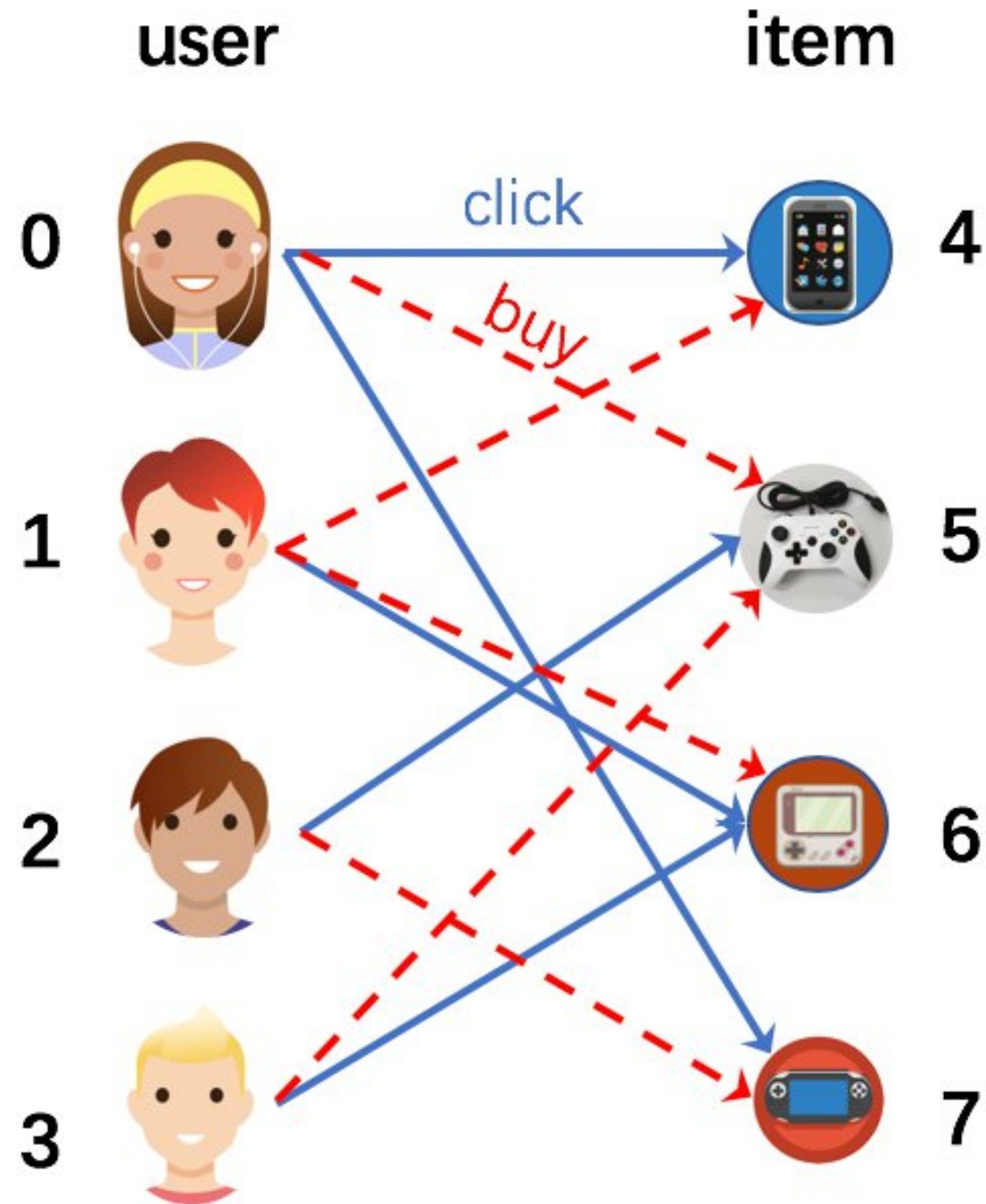


Классификация: Выдавать/не выдавать кредитку

Регрессия: Степень интереса в тематике

Кластеризация: Выделение комьюнити

Edge level

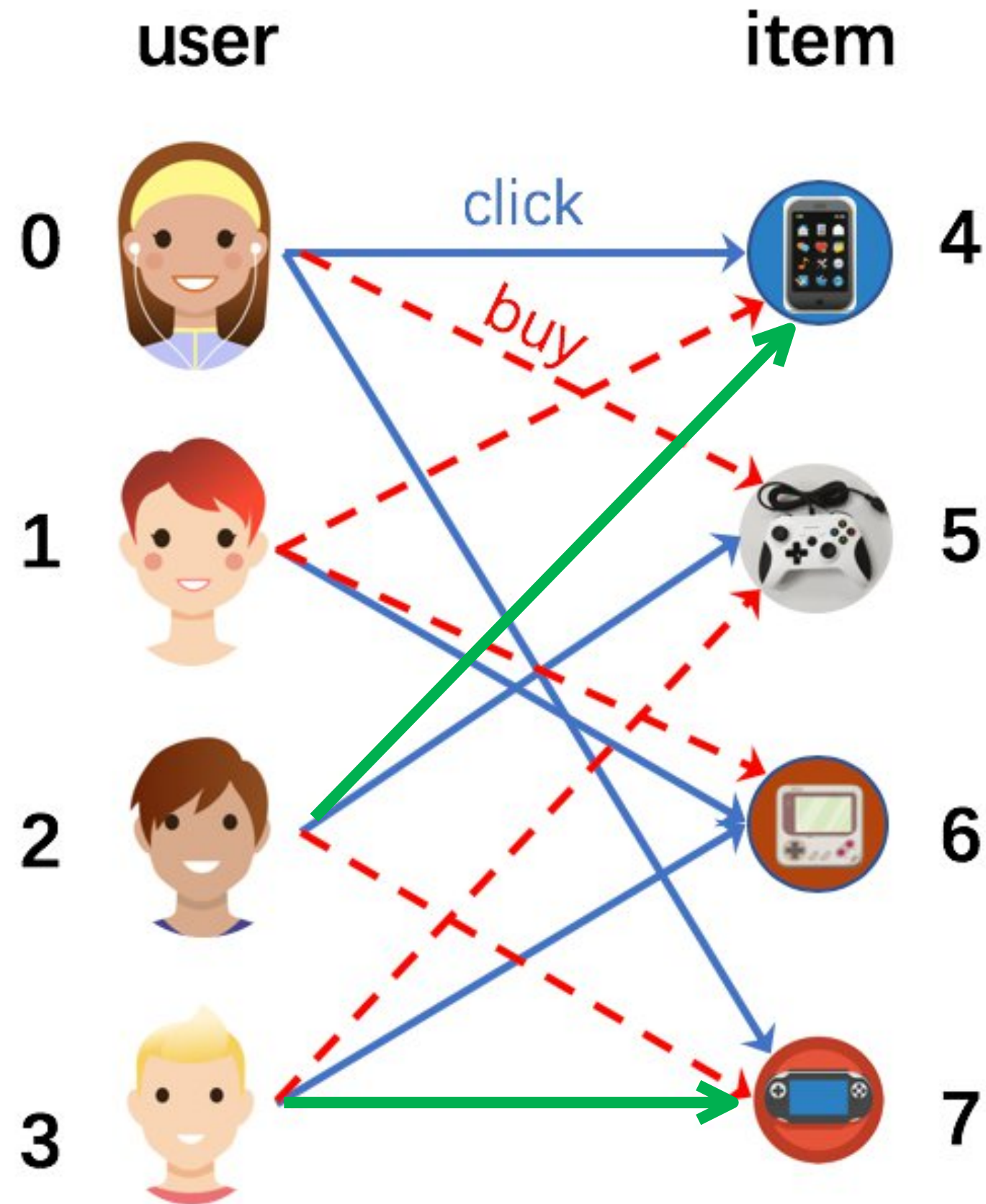


Классификация: Определение типа взаимодействия

Регрессия: Степень интереса к товару

Дополнение графа: Предсказания взаимодействия

Edge level



Классификация: Определение типа взаимодействия

Регрессия: Степень интереса к товару

Дополнение графа: Предсказания взаимодействия

user-user graph

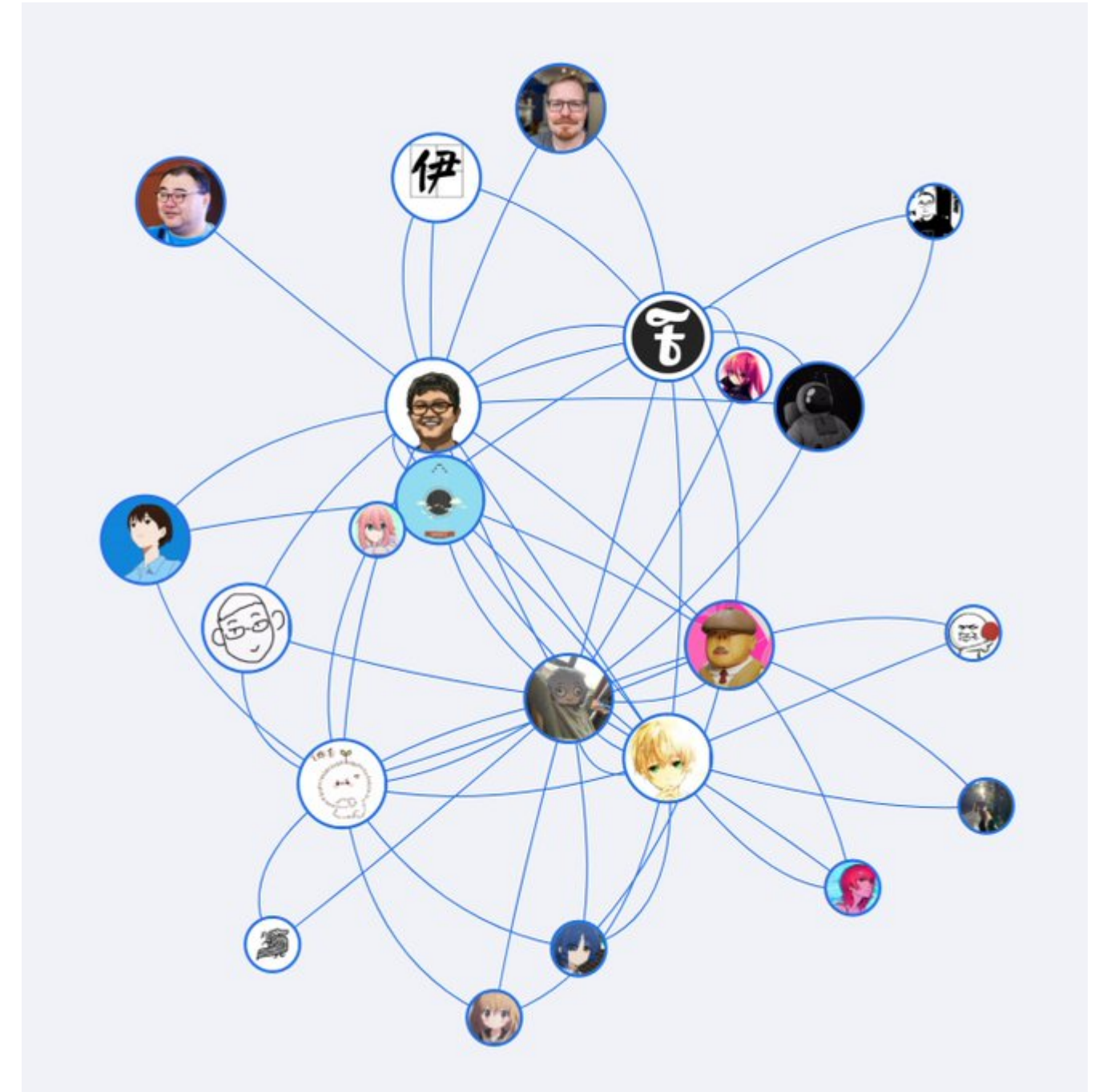
$G = (U, E)$, где

U - множество пользователей

E - множество ребер, отображающих взаимодействия либо свойства похожести между пользователями

Такой граф называется социальным.

Пример задачи: рекомендация новых друзей.



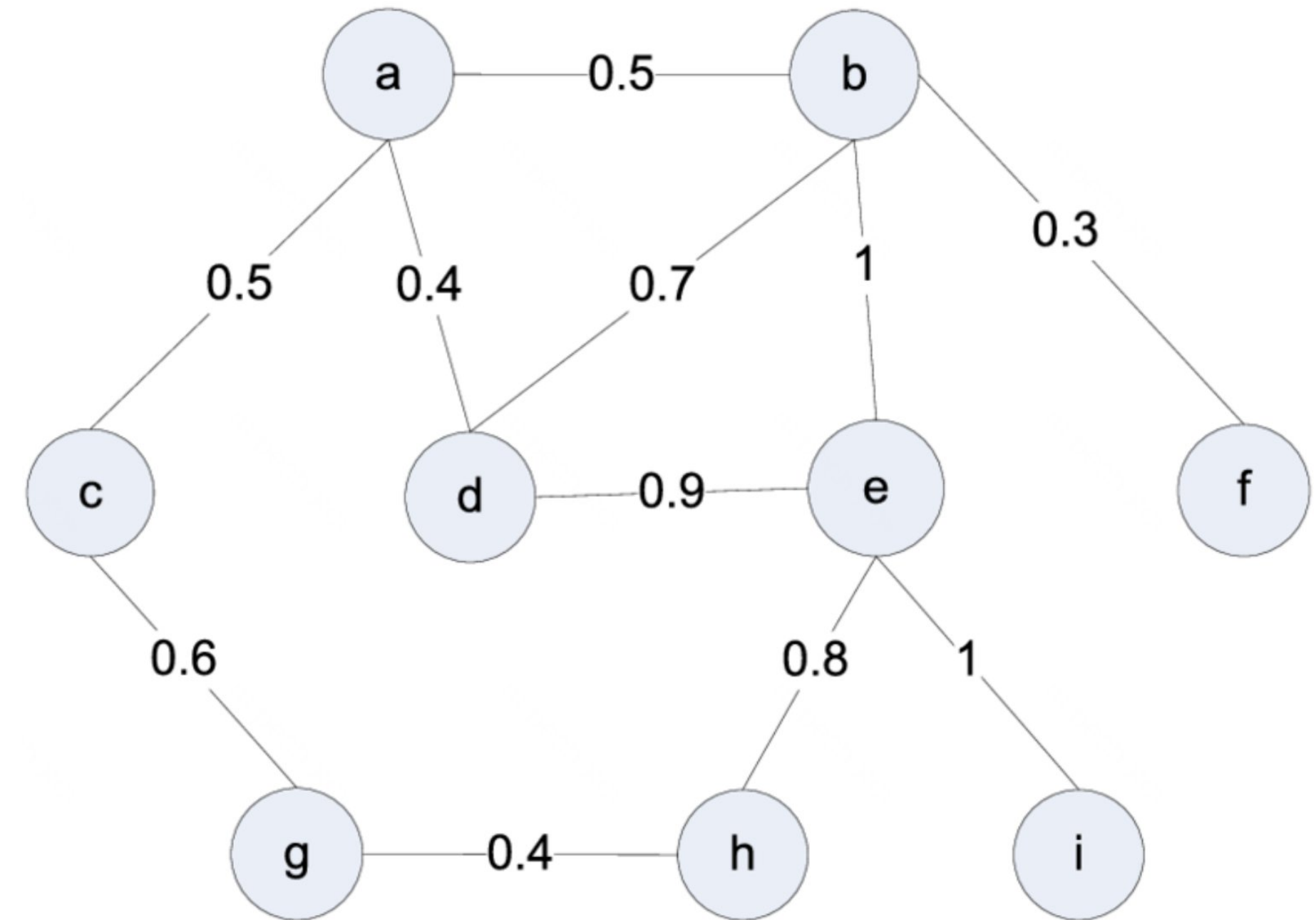
item-item graph

$G = (I, E)$, где

I - множество айтемов

E - множество ребер, отображающих взаимодействия либо свойства похожести между айтемами

Пример задачи: рекомендация похожих/сопутствующих товаров.



user-item graph

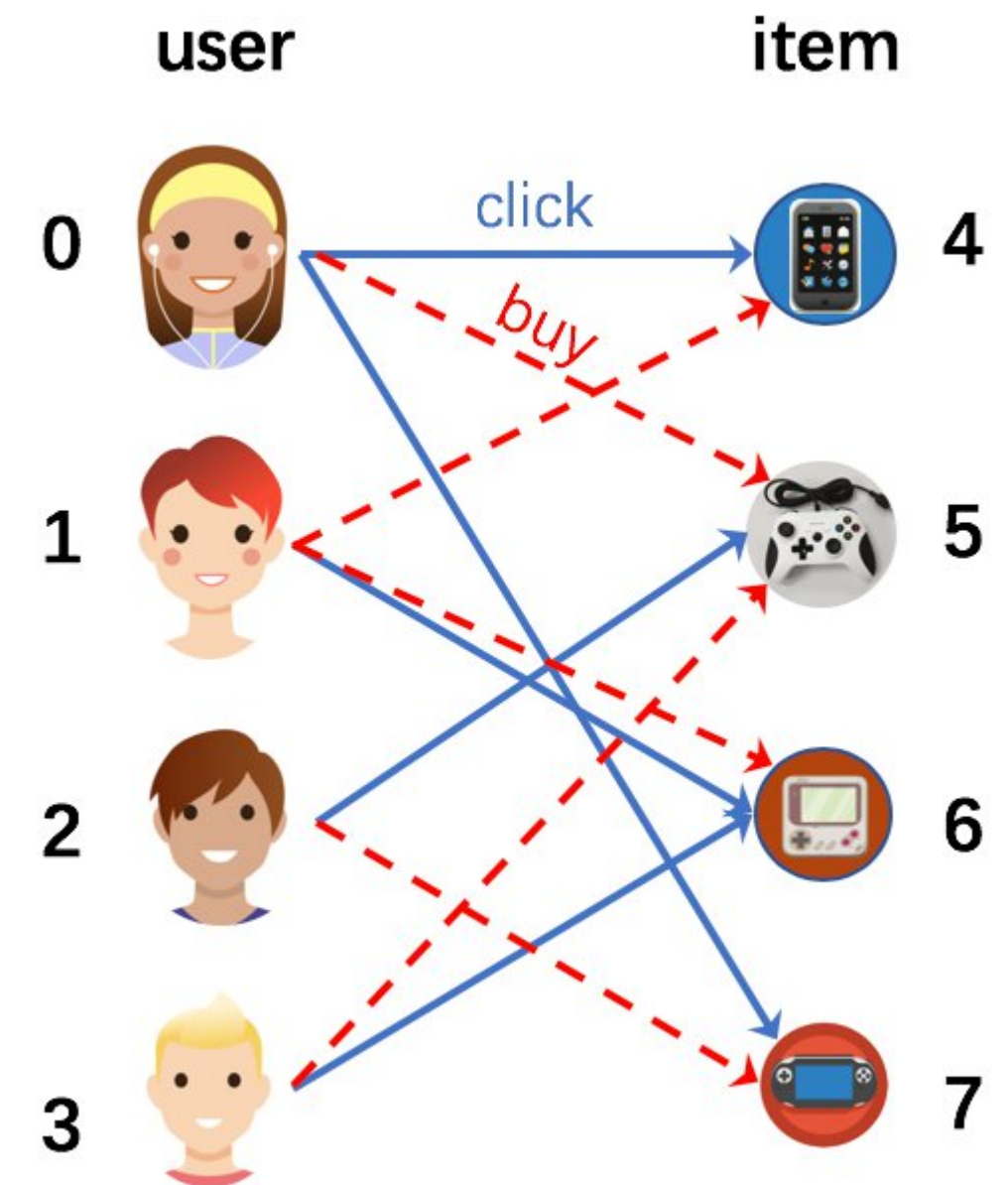
$G = (V, E)$, где

V - множество юзеров и айтемов

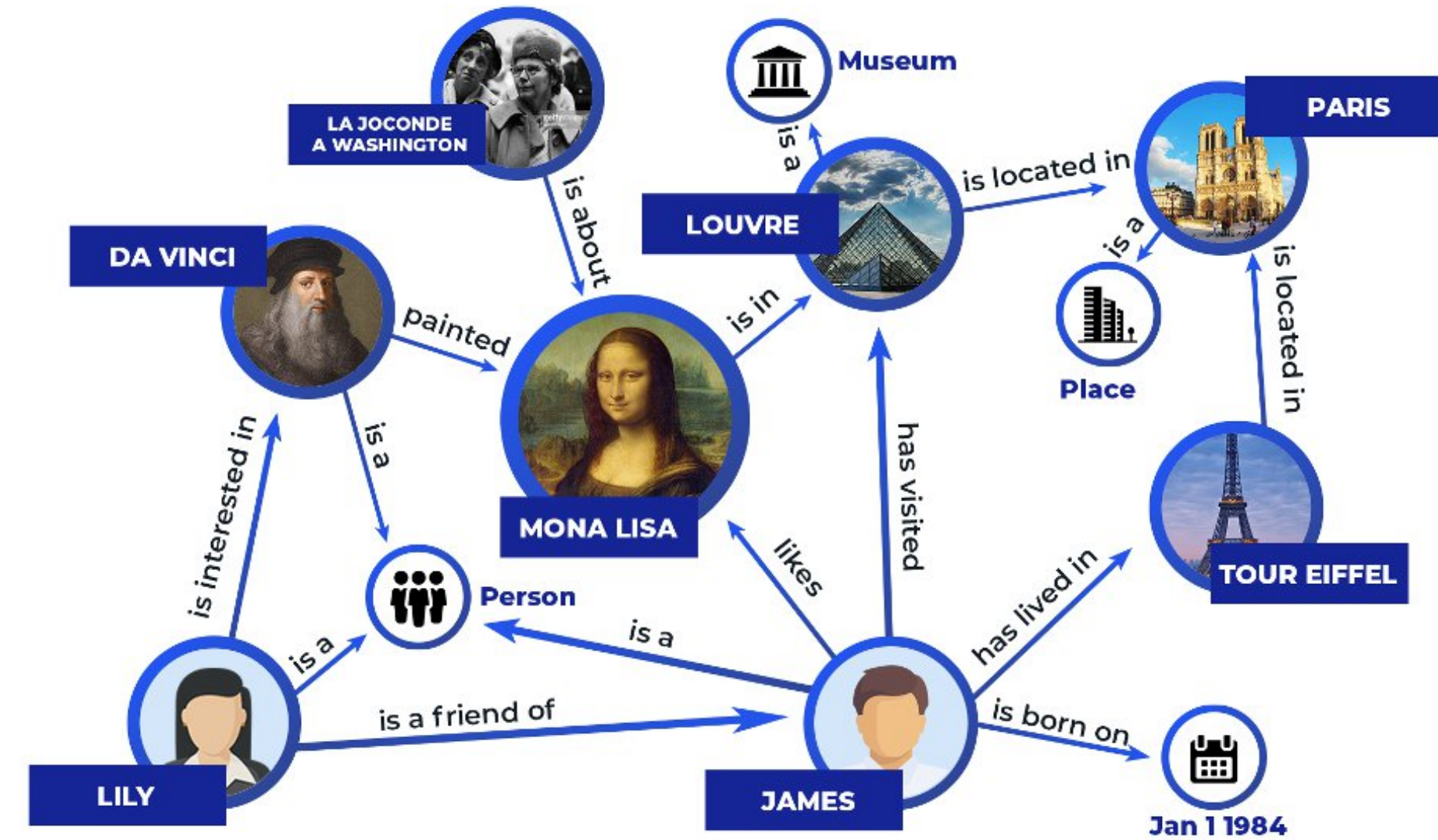
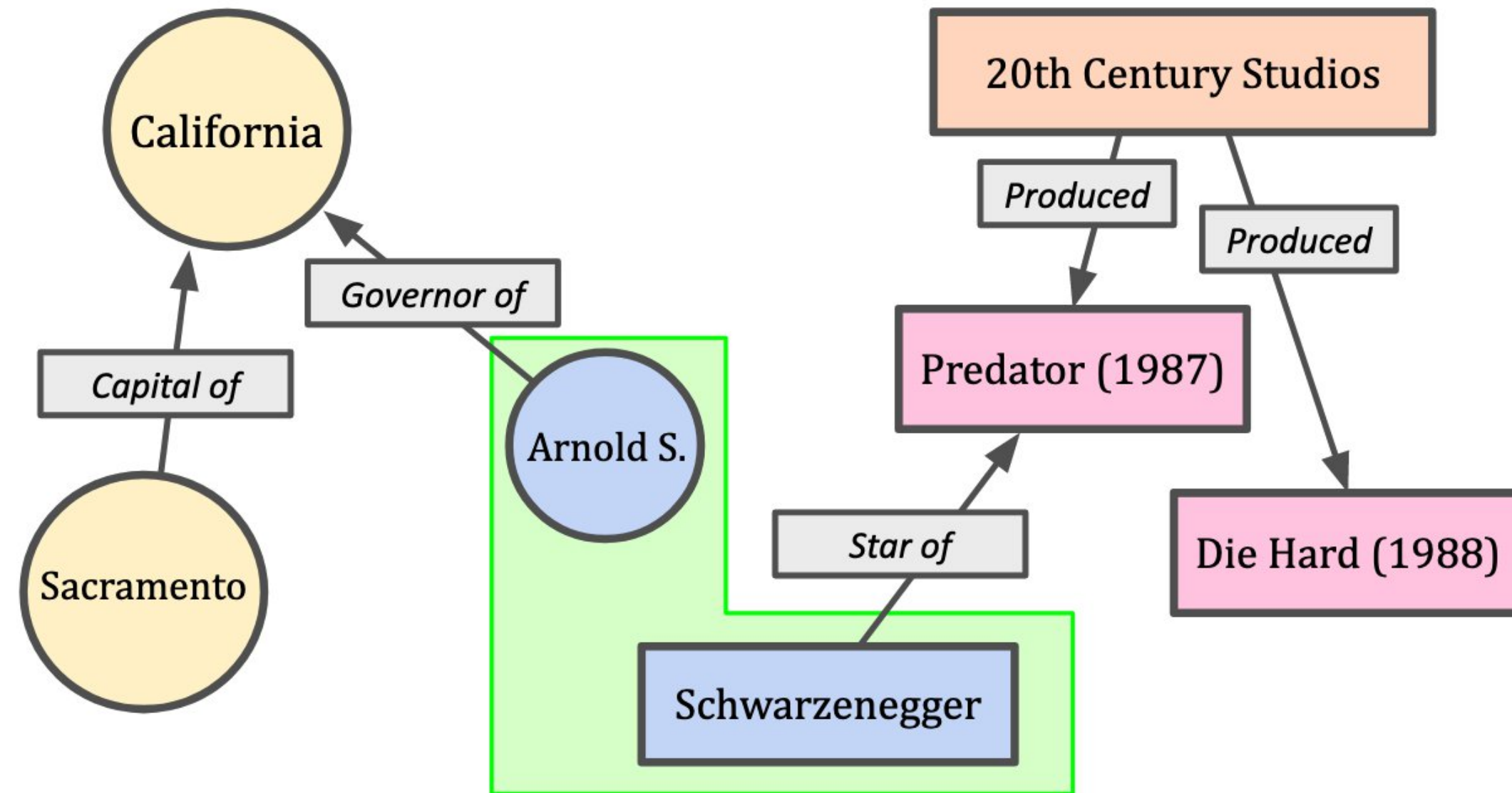
E - множество ребер, отображающих взаимодействия между пользователями и айтемами

Граф, как правило, является двудольным

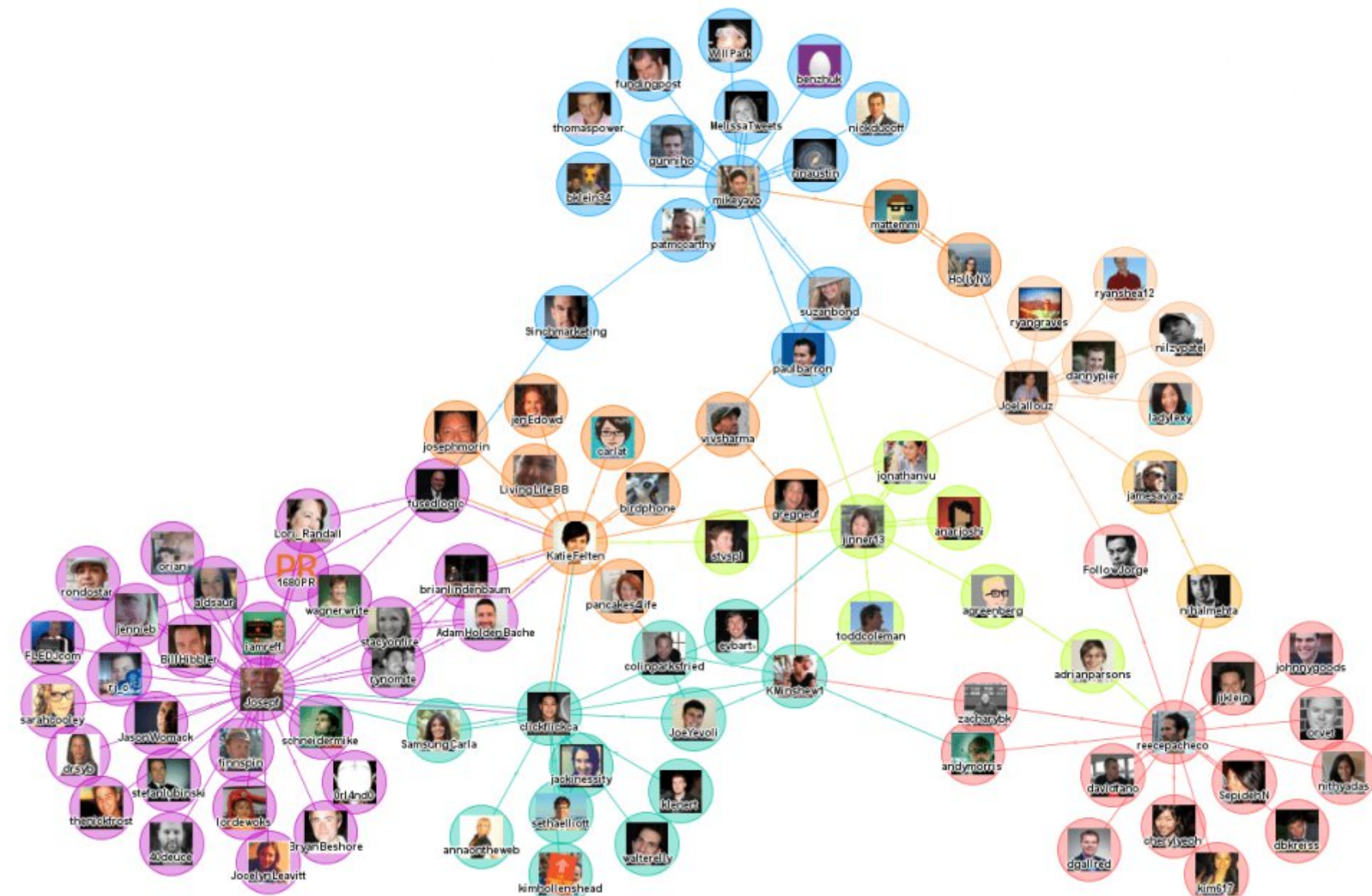
Пример задачи: рекомендации фильмов/товаров



knowledge graph

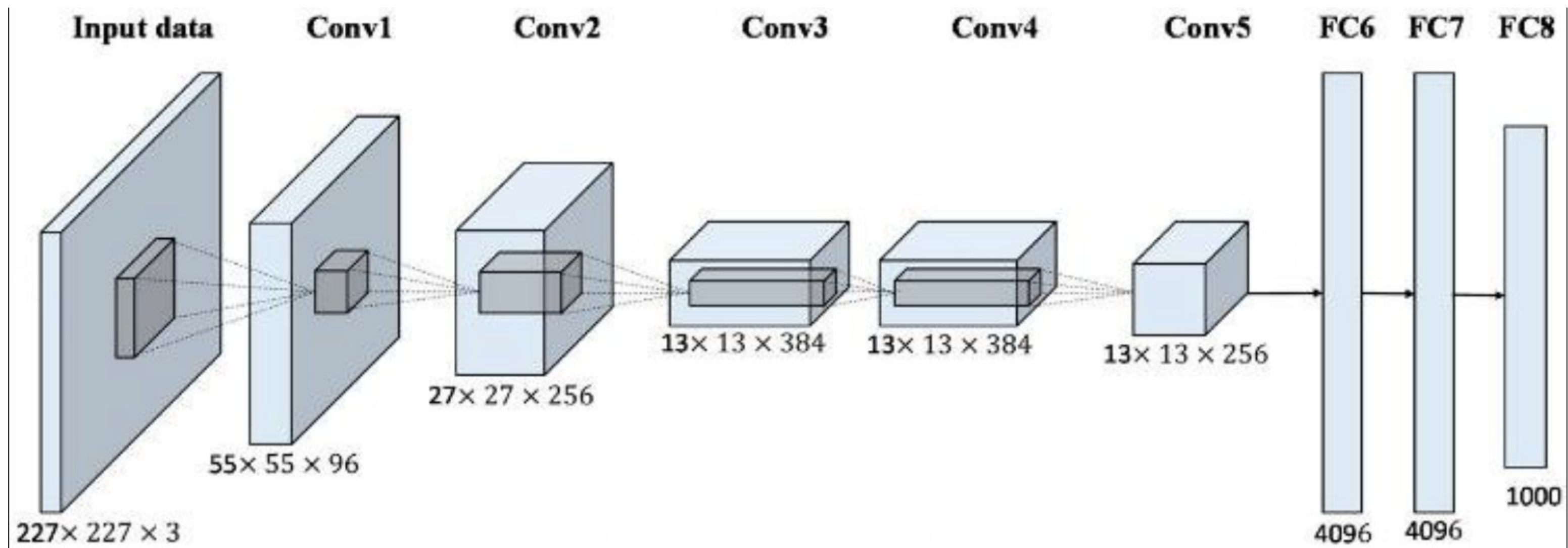


Московский
СКОЛКОВО



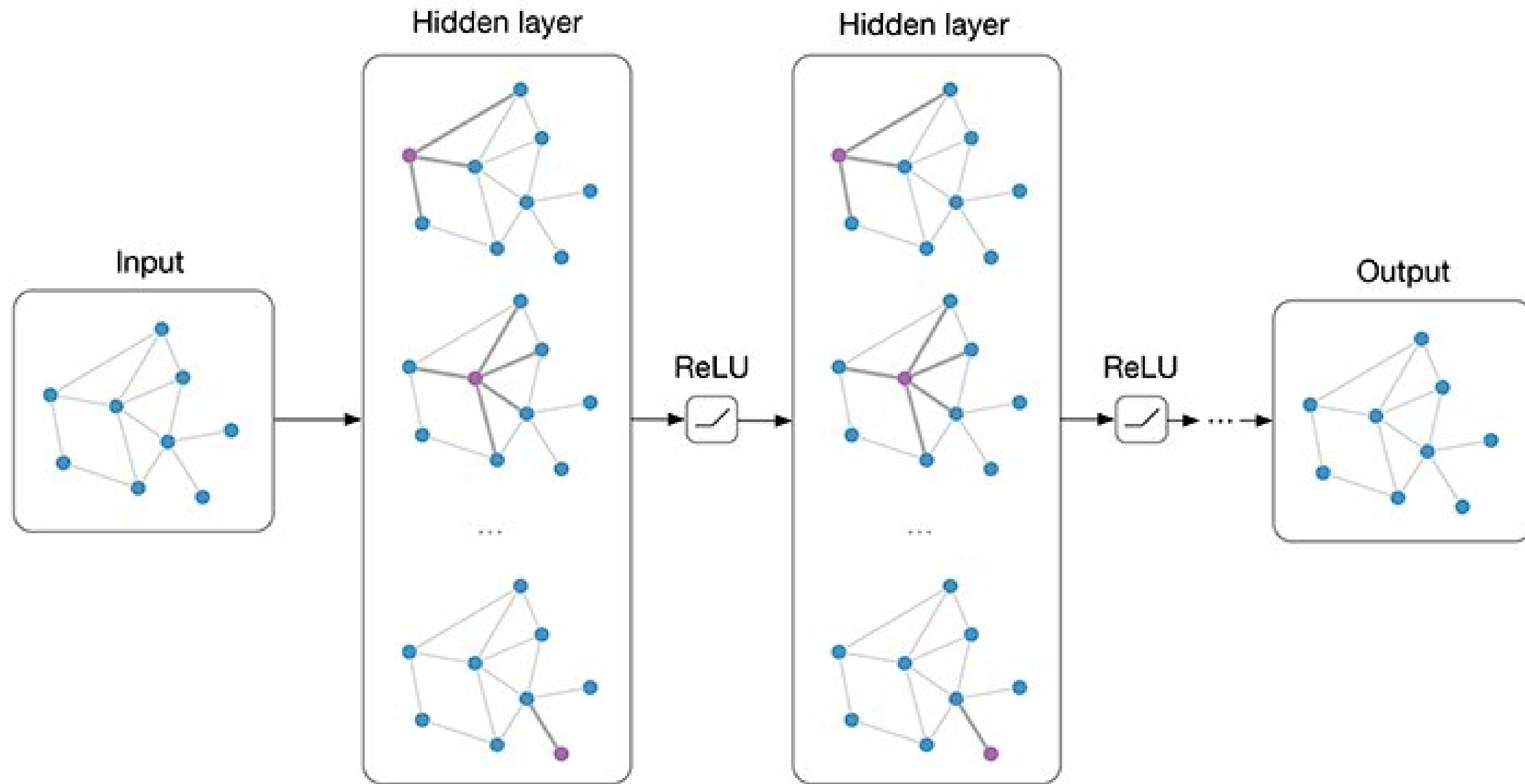
Графовые свертки

Графовые свертки



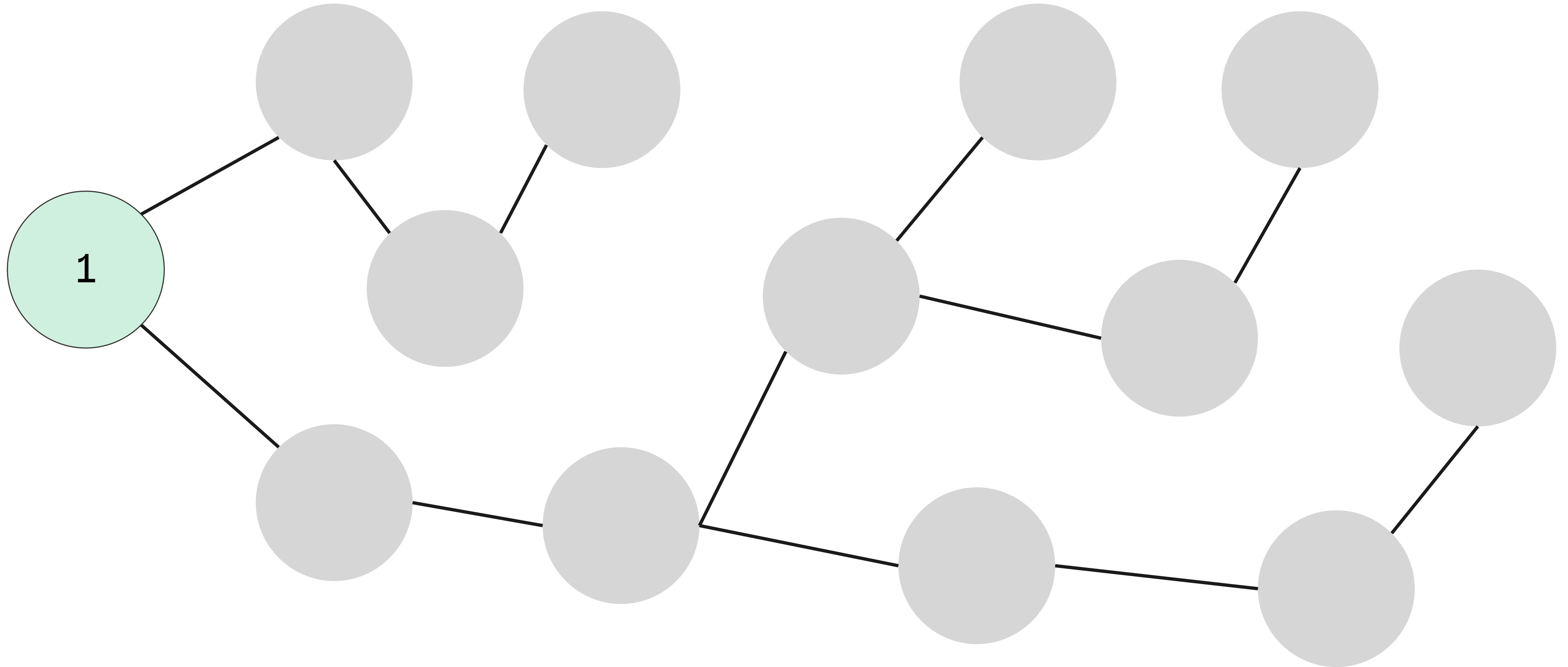
AlexNet

Графовые свертки

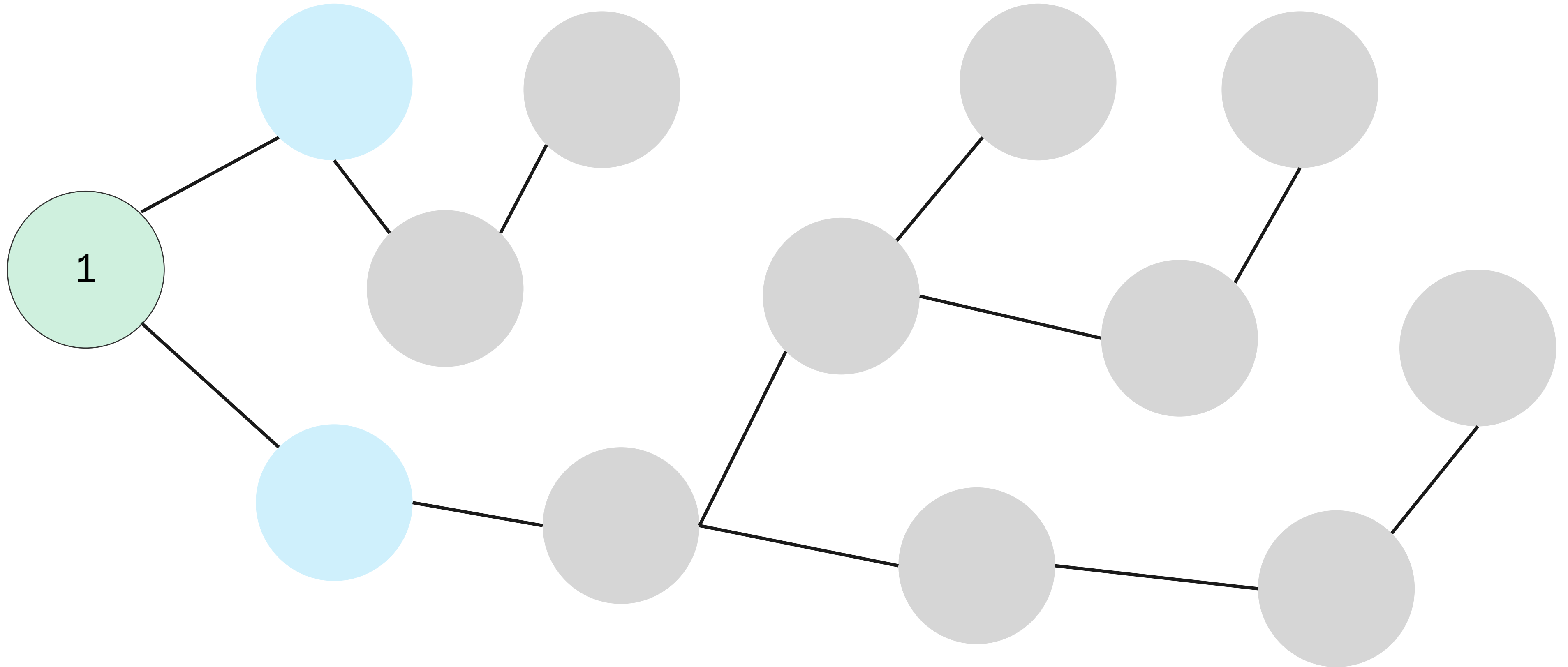


графовая свертка

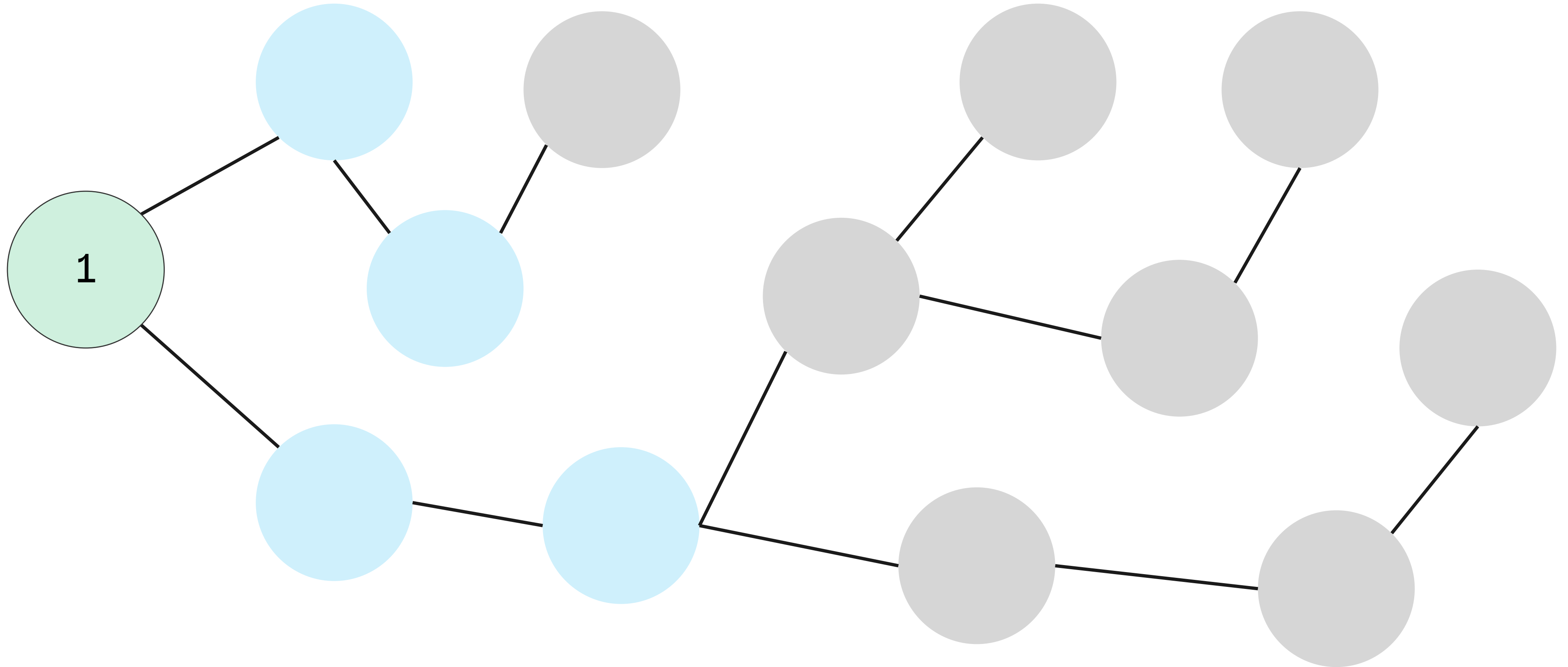
Графовые свертки. Receptive field



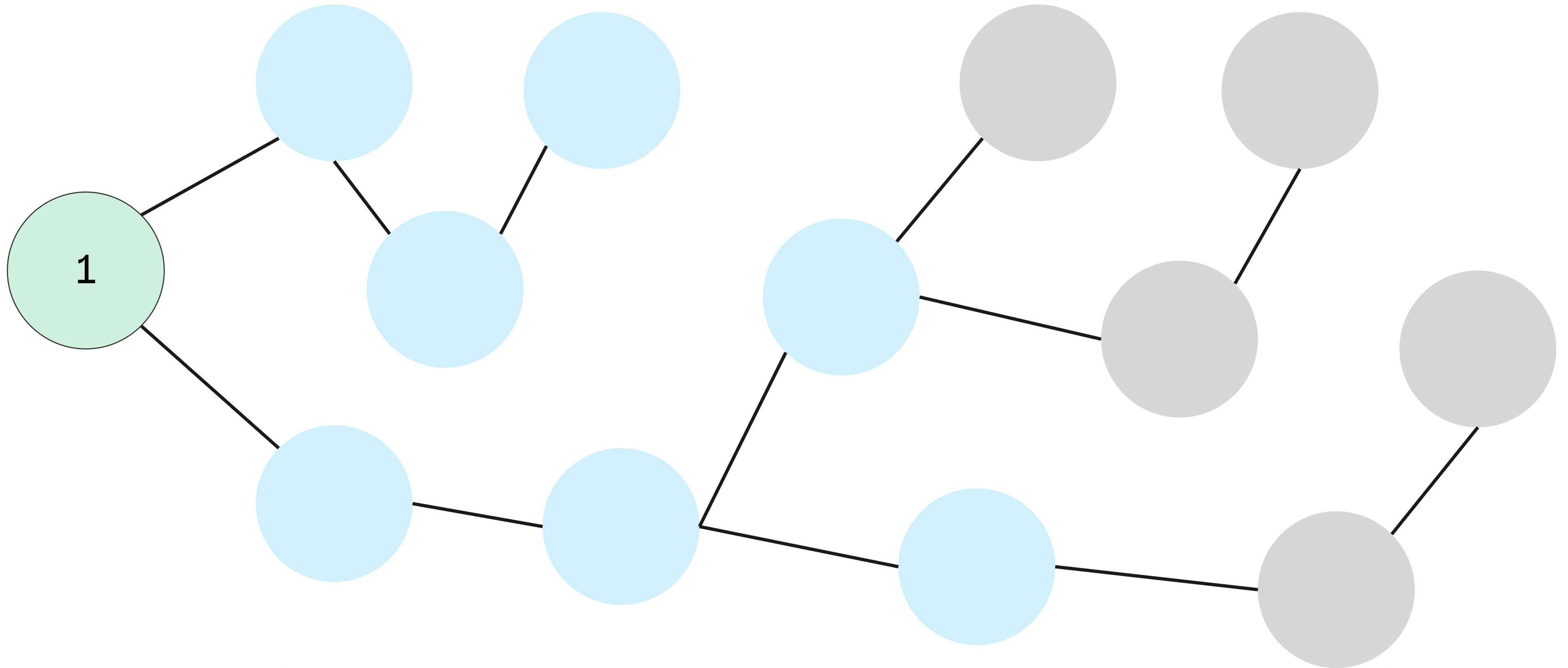
Графовые свертки. Receptive field



Графовые свертки. Receptive field

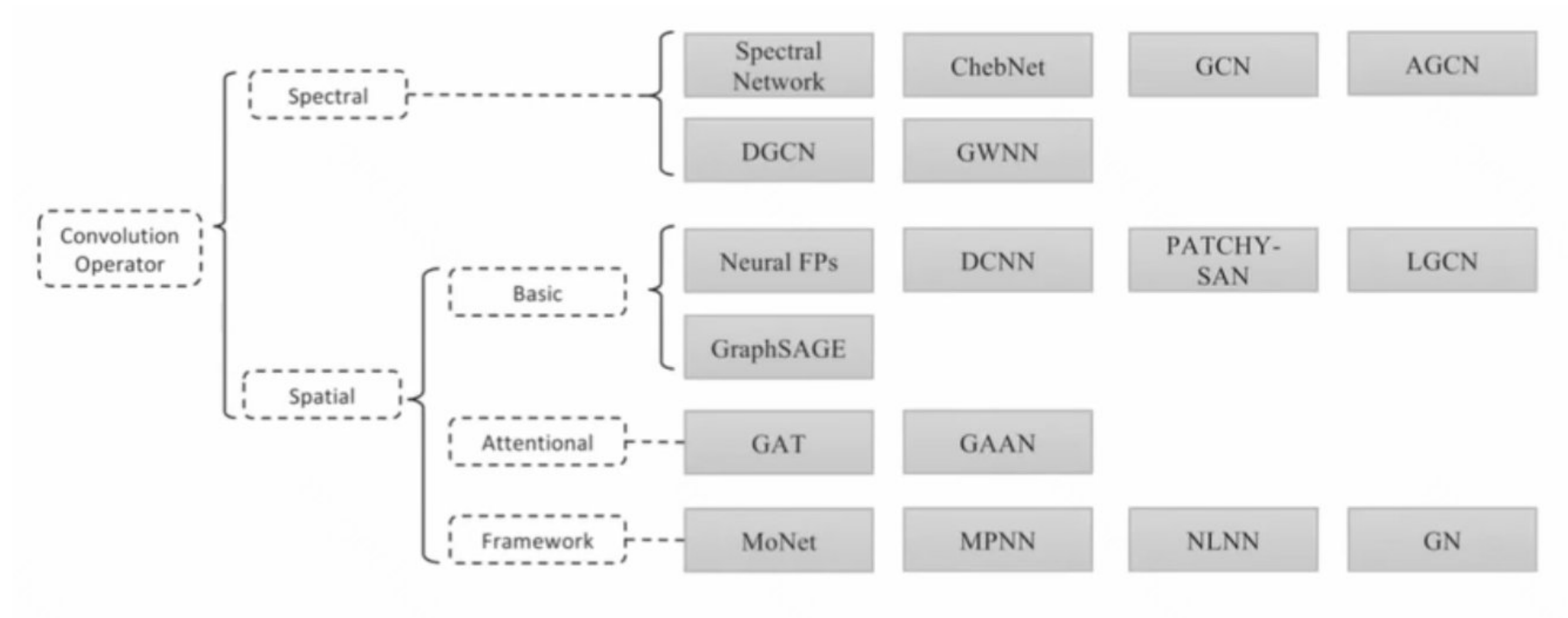


Графовые свертки. Receptive field



N сверток собирают информацию с соседей, до которых можно дойти не больше чем за N шагов

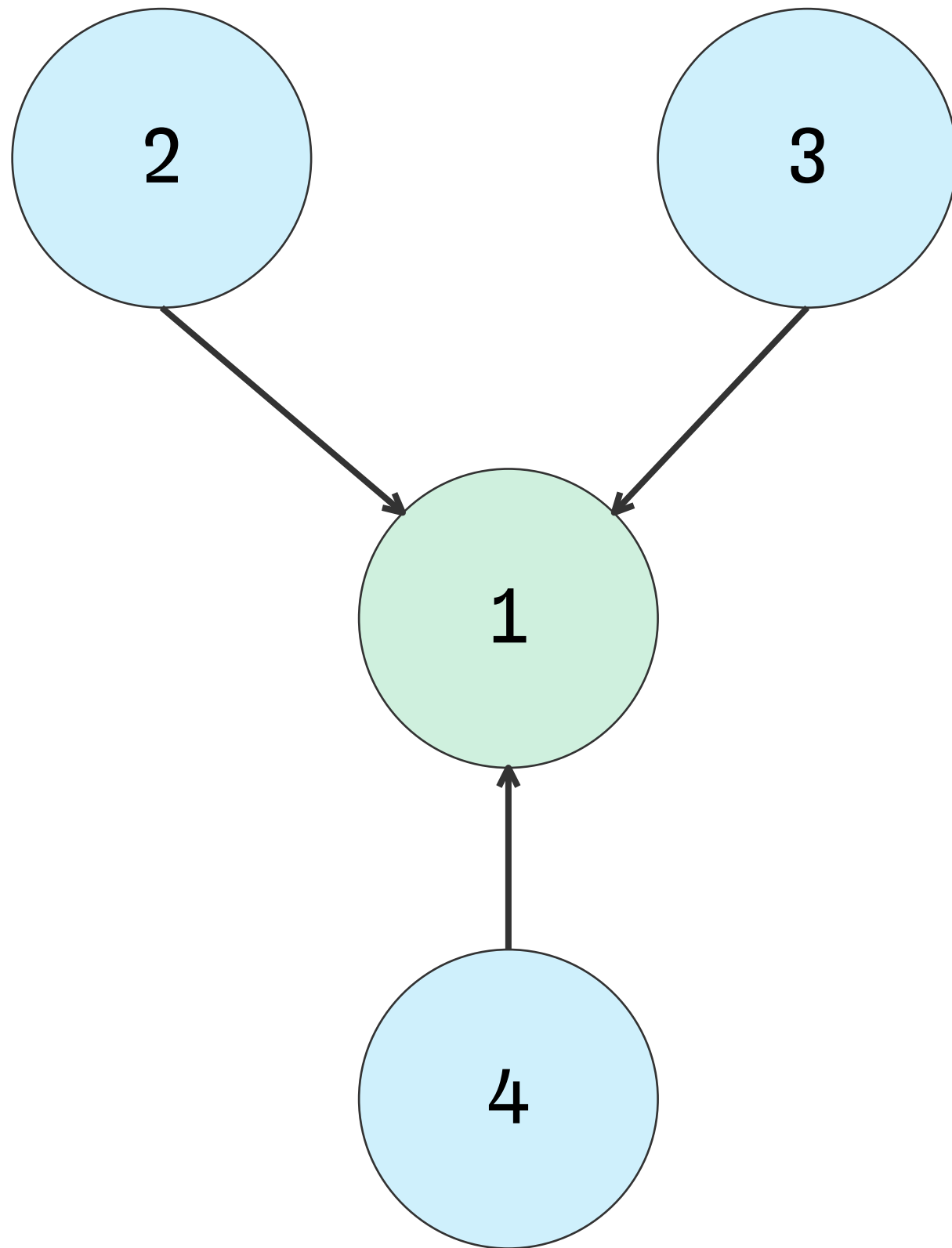
Графовые свертки. Парадигмы



Парадигмы построения сверток:

- **передача сообщений(локальная)**
 - использует локальную окрестность вершины
- **спектральная парадигма(глобальная)**
 - использует глобальное положение вершины в графе

Графовые свертки. Передача сообщений



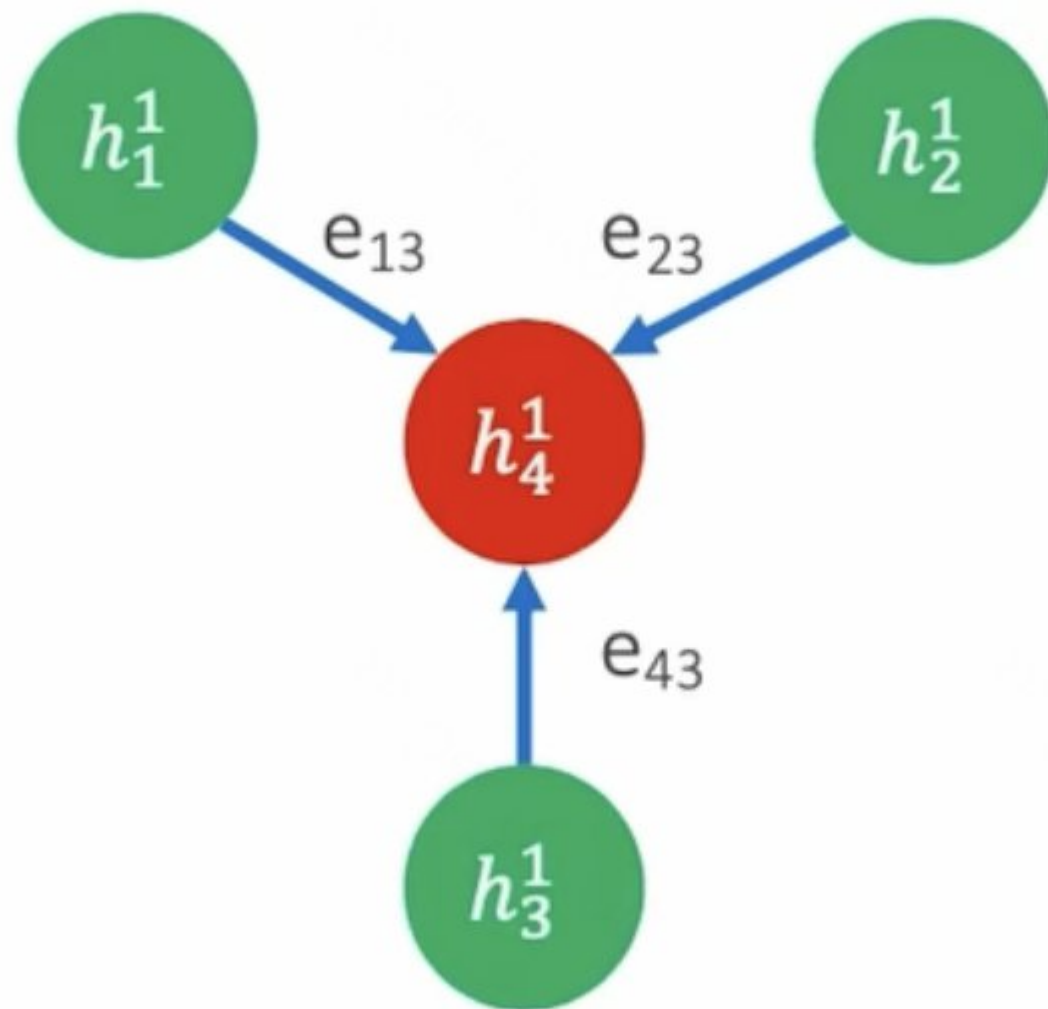
1. Подсчет сообщения

$$m_v^{t+1} = \sum_{w \in N(v)} M_t(h_v^t, h_w^t, e_{wv})$$

2. Обновление состояния

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

Графовые свертки. Передача сообщений



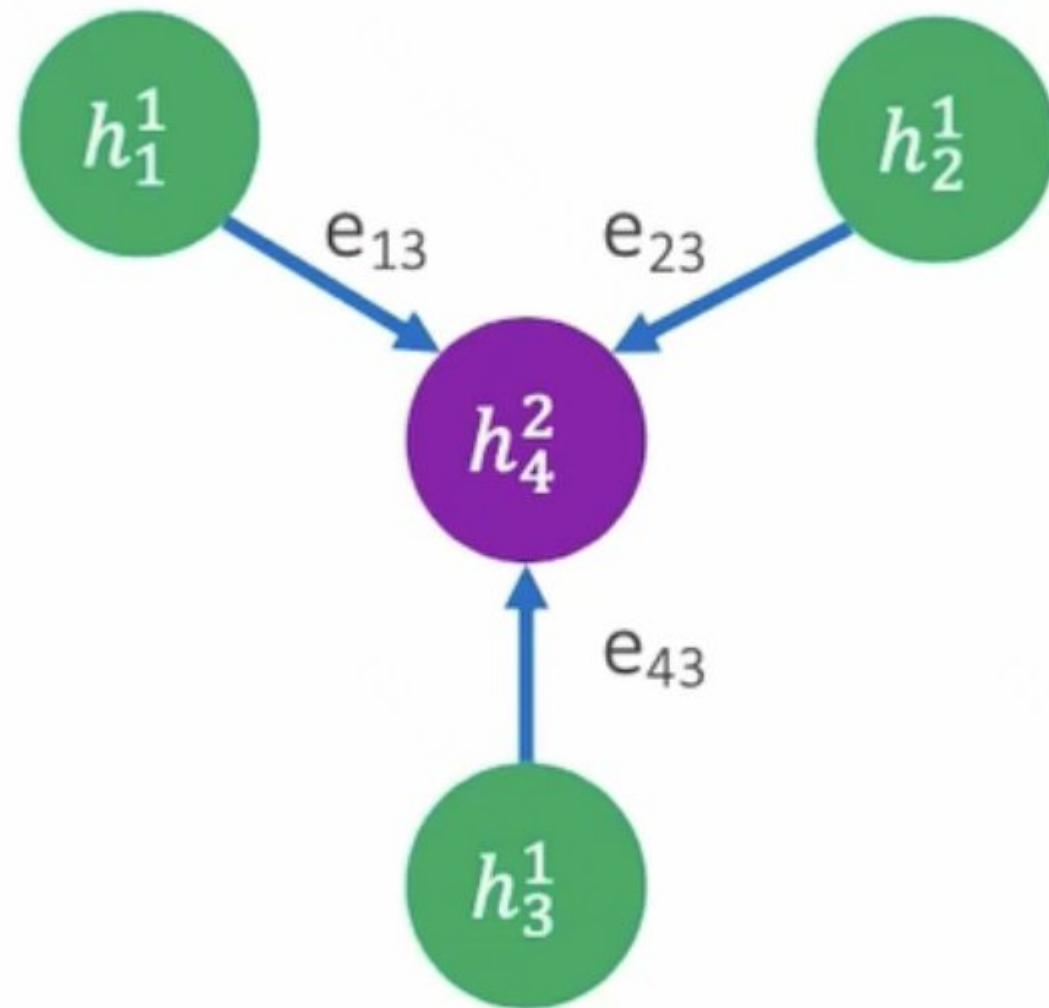
1. Подсчет сообщения

$$m_v^{t+1} = \sum_{w \in N(v)} M_t(h_v^t, h_w^t, e_{wv})$$

2. Обновление состояния

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

Графовые свертки. Передача сообщений



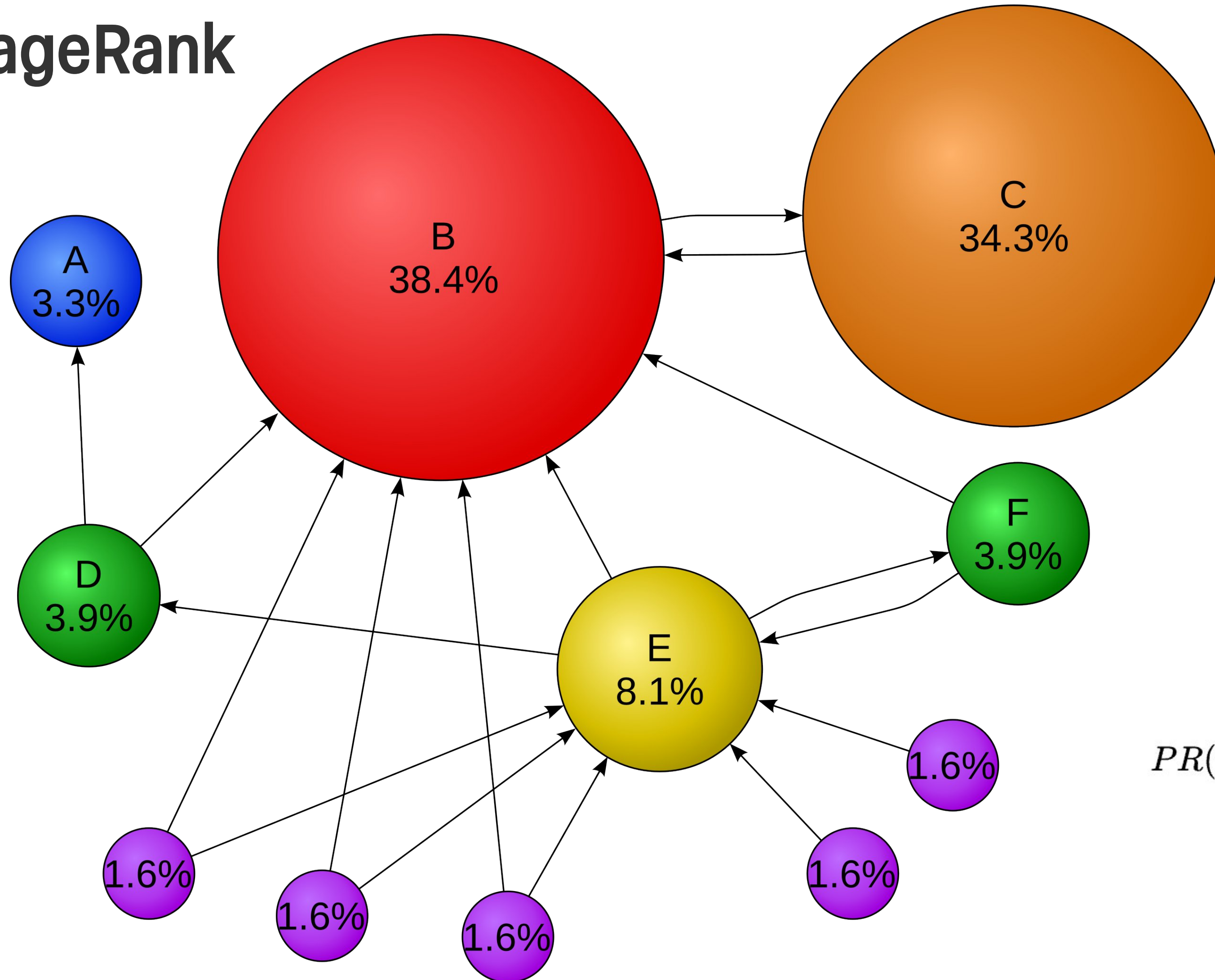
1. Подсчет сообщения

$$m_4^2 = M_t(h_1^1, h_4^1, e_{14}) + M_t(h_2^1, h_4^1, e_{24}) + M_t(h_3^1, h_4^1, e_{34})$$

2. Обновление состояния

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

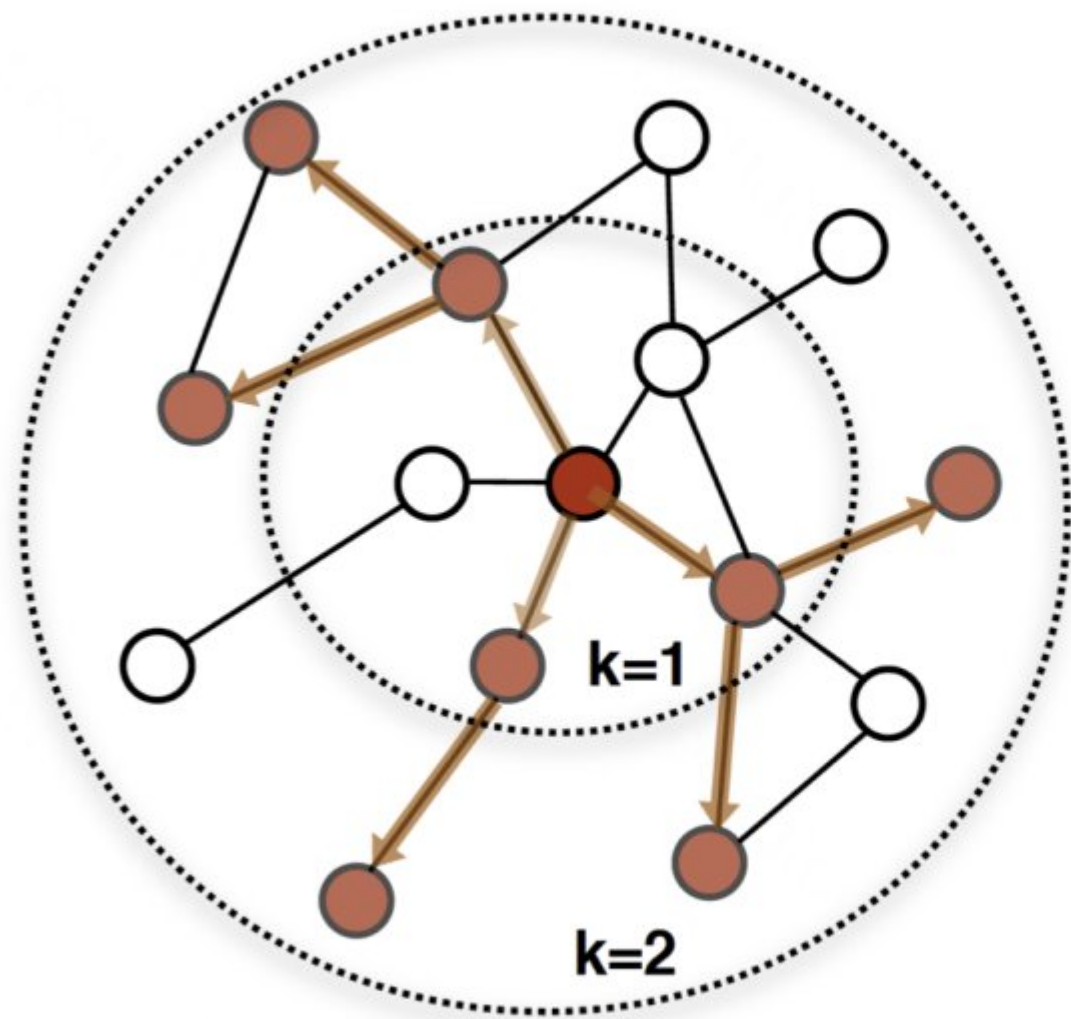
PageRank



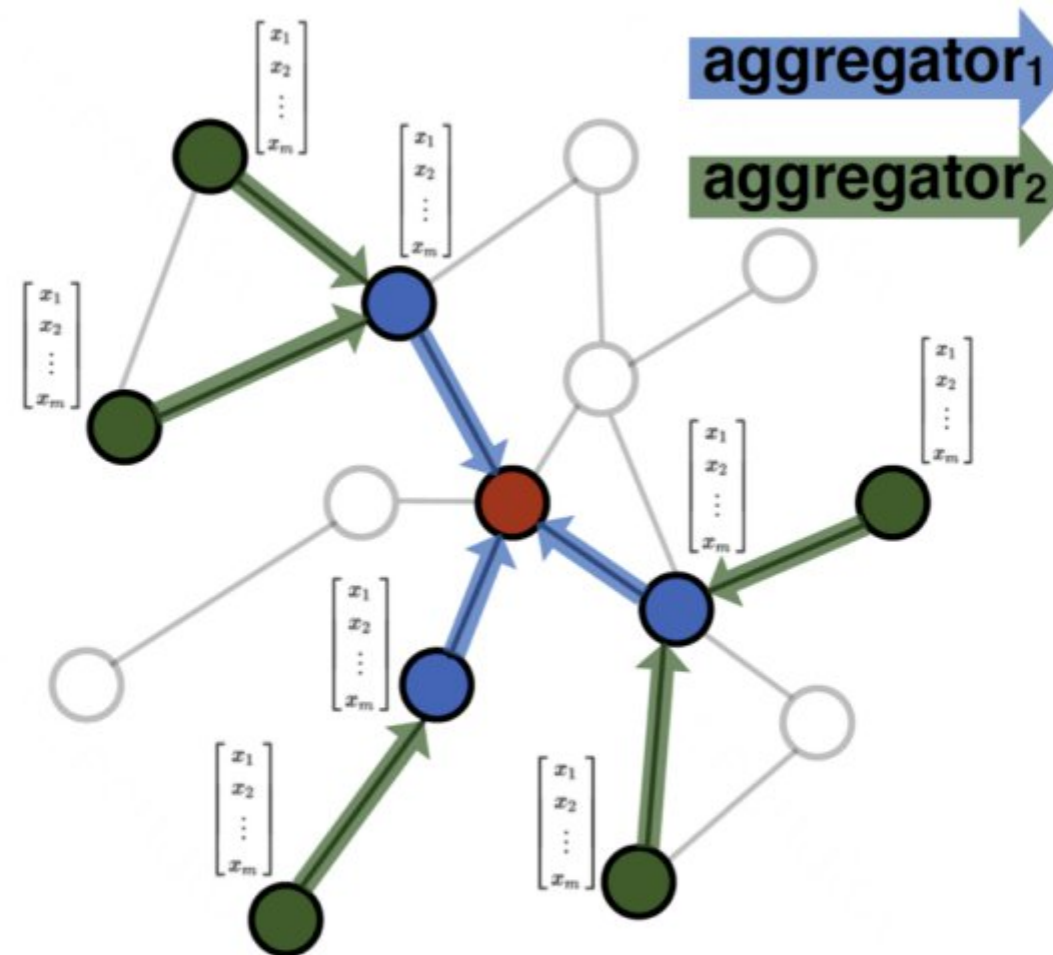
$$PR(u) = \sum_{v \in B_u} \frac{PR(v)}{L(v)},$$

$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

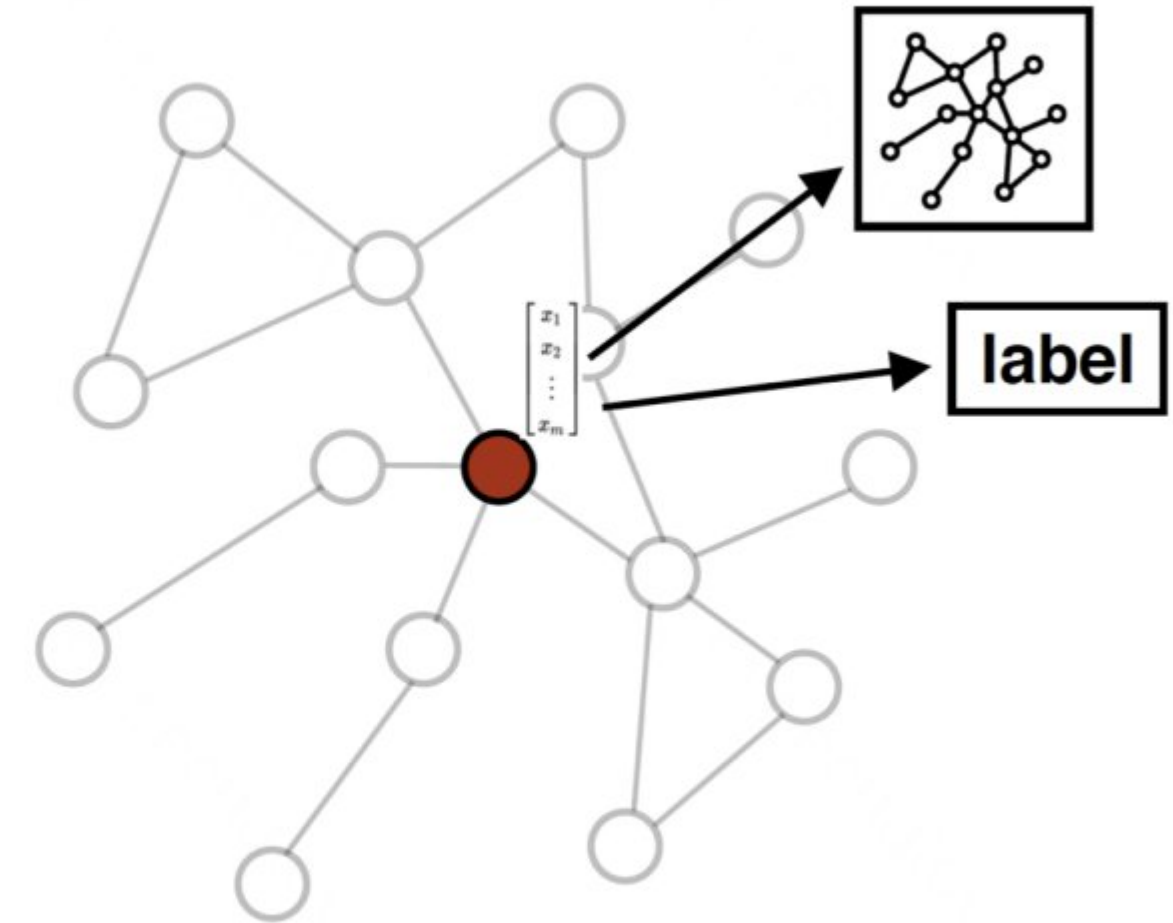
GraphSage



1. Sample neighborhood



2. Aggregate feature information from neighbors



3. Predict graph context and label using aggregated information

сообщение

$$\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\});$$

$$\mathbf{h}_v^k \leftarrow \sigma \left(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k) \right)$$

обучаемая матрица

GraphSage

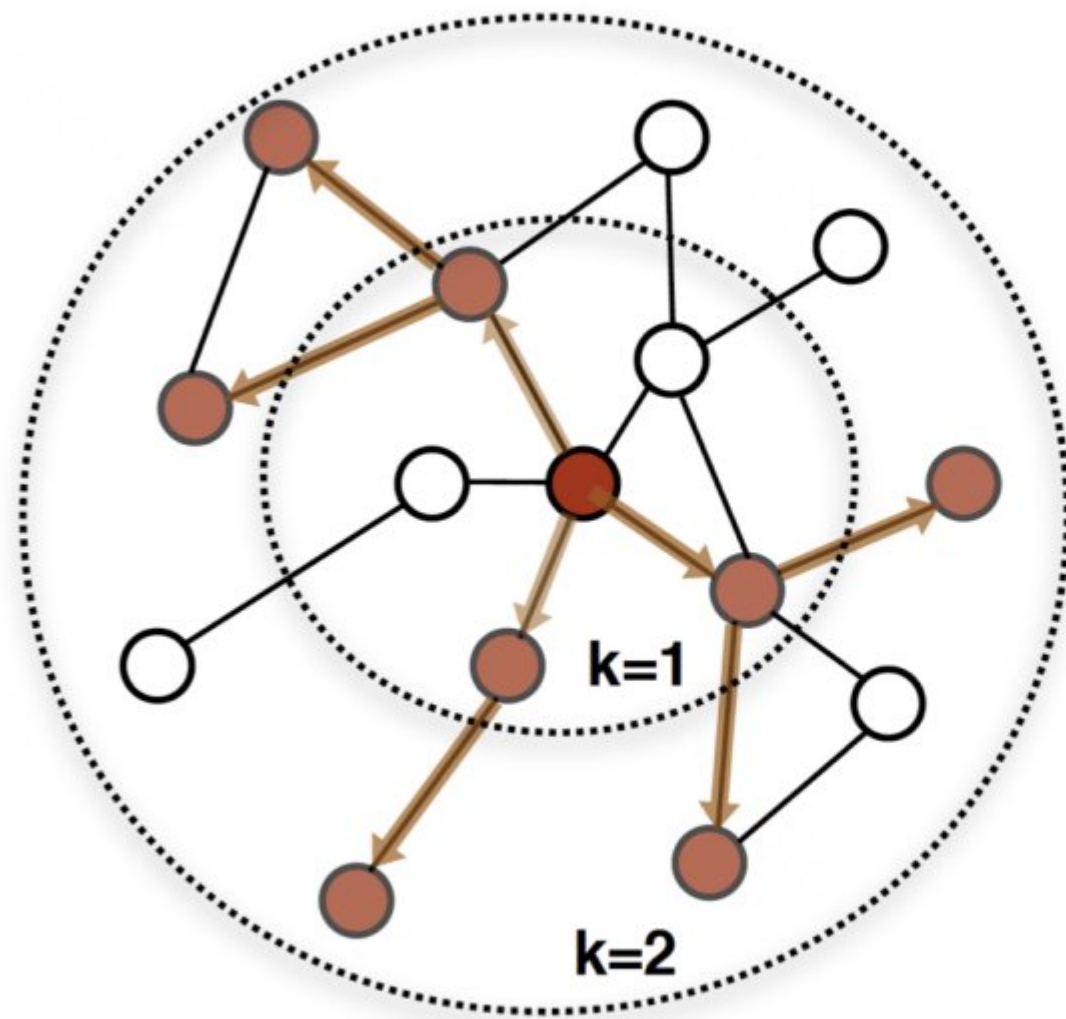
Algorithm 1: GraphSAGE embedding generation (i.e., forward propagation) algorithm

Input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input features $\{\mathbf{x}_v, \forall v \in \mathcal{V}\}$; depth K ; weight matrices $\mathbf{W}^k, \forall k \in \{1, \dots, K\}$; non-linearity σ ; differentiable aggregator functions $\text{AGGREGATE}_k, \forall k \in \{1, \dots, K\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

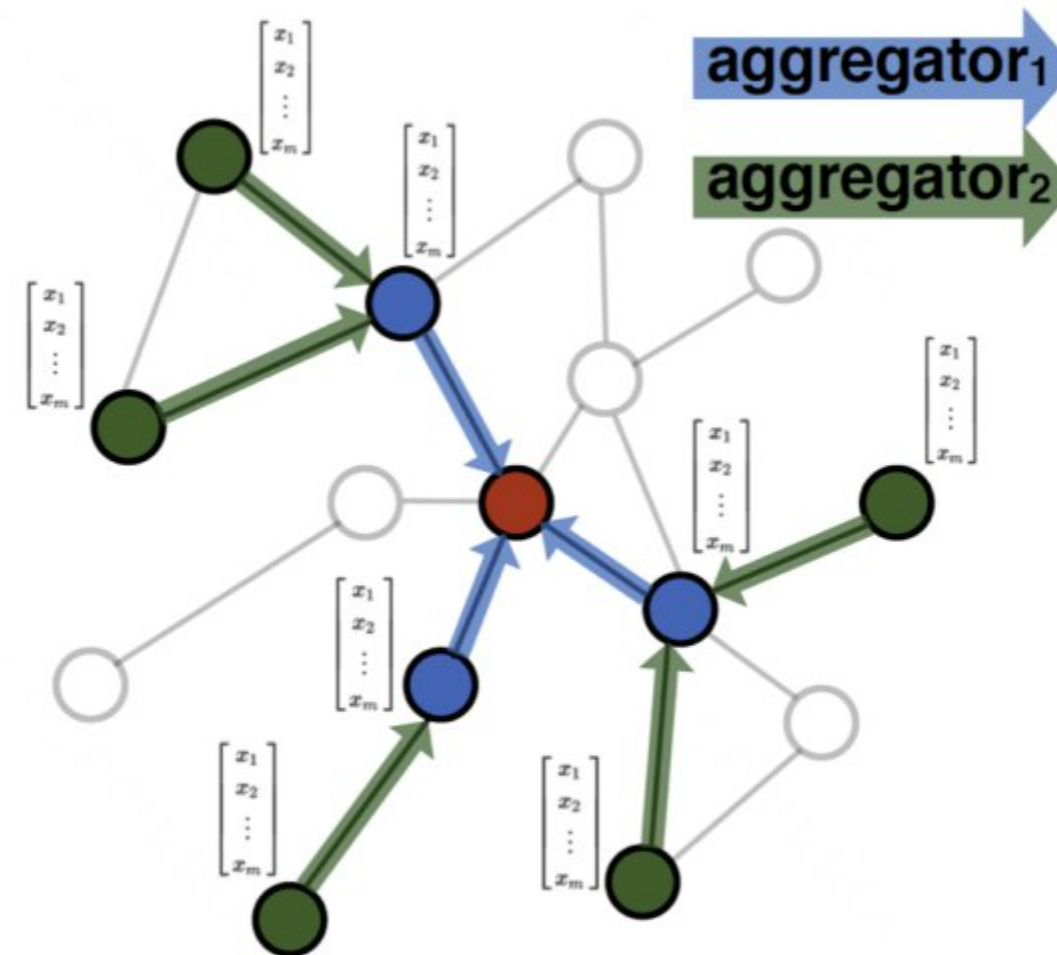
Output : Vector representations $\mathbf{z}_v$ for all $v \in \mathcal{V}$

```
1  $\mathbf{h}_v^0 \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$  ;
2 for  $k = 1 \dots K$  do
3   for  $v \in \mathcal{V}$  do
4      $\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\})$ ;
5      $\mathbf{h}_v^k \leftarrow \sigma \left( \mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k) \right)$ 
6   end
7    $\mathbf{h}_v^k \leftarrow \mathbf{h}_v^k / \|\mathbf{h}_v^k\|_2, \forall v \in \mathcal{V}$ 
8 end
9  $\mathbf{z}_v \leftarrow \mathbf{h}_v^K, \forall v \in \mathcal{V}$ 
```

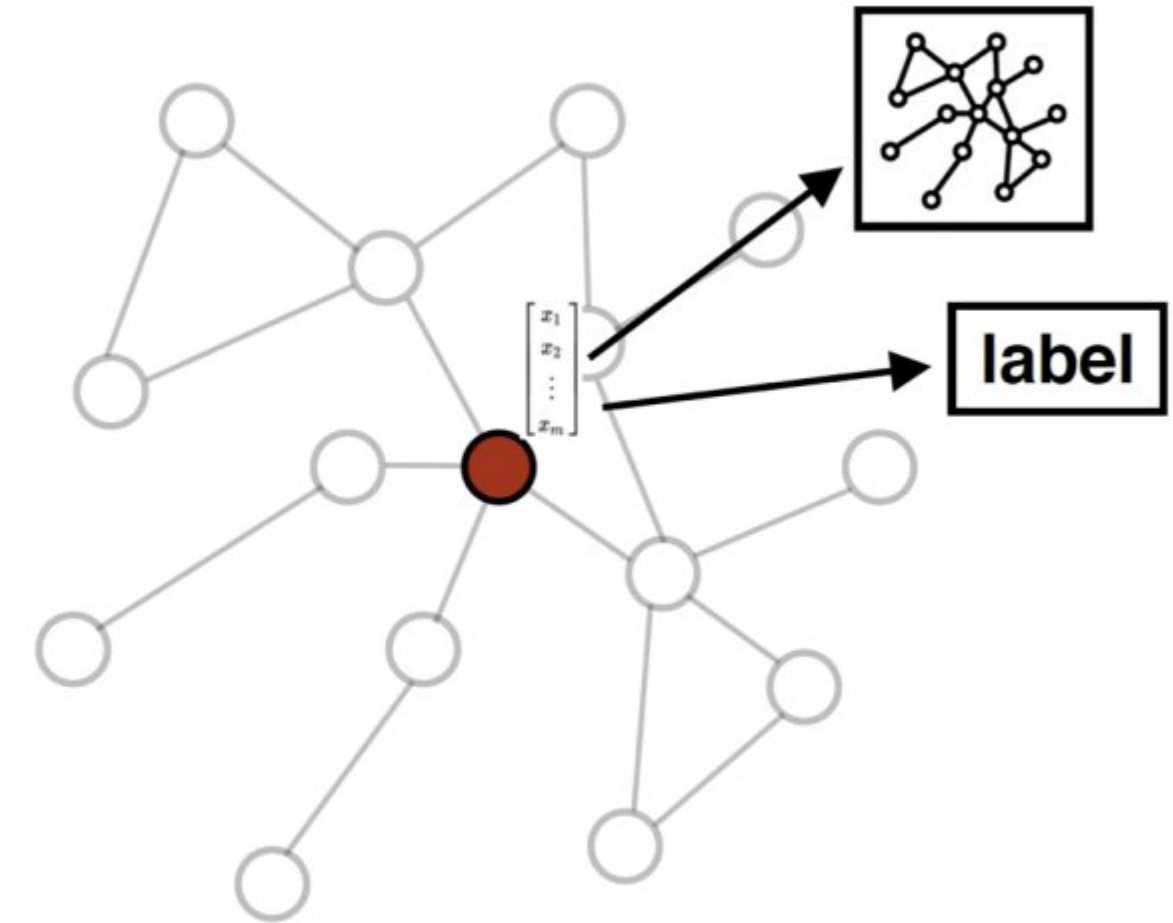
GraphSage



1. Sample neighborhood



2. Aggregate feature information from neighbors



3. Predict graph context and label using aggregated information

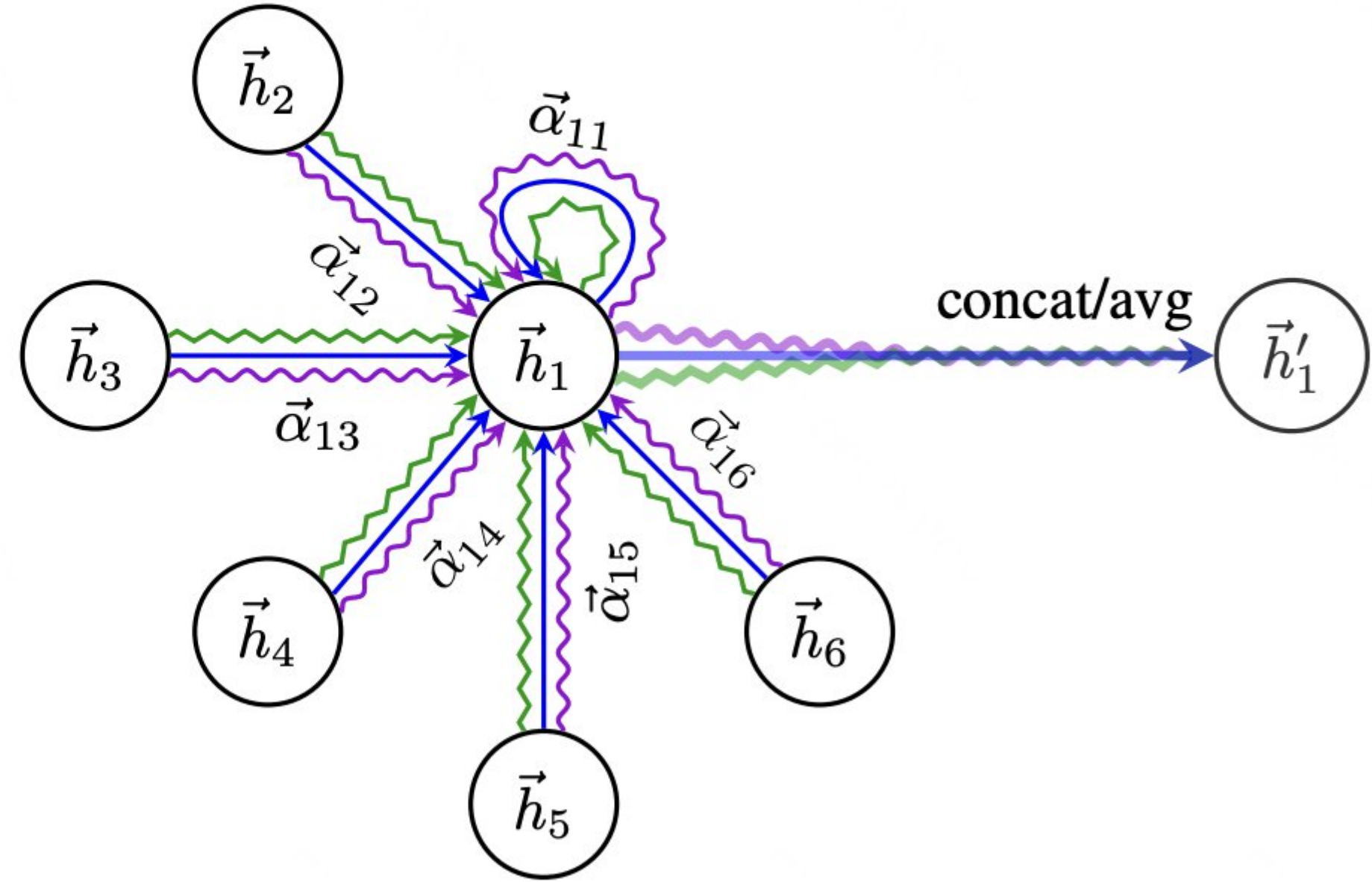
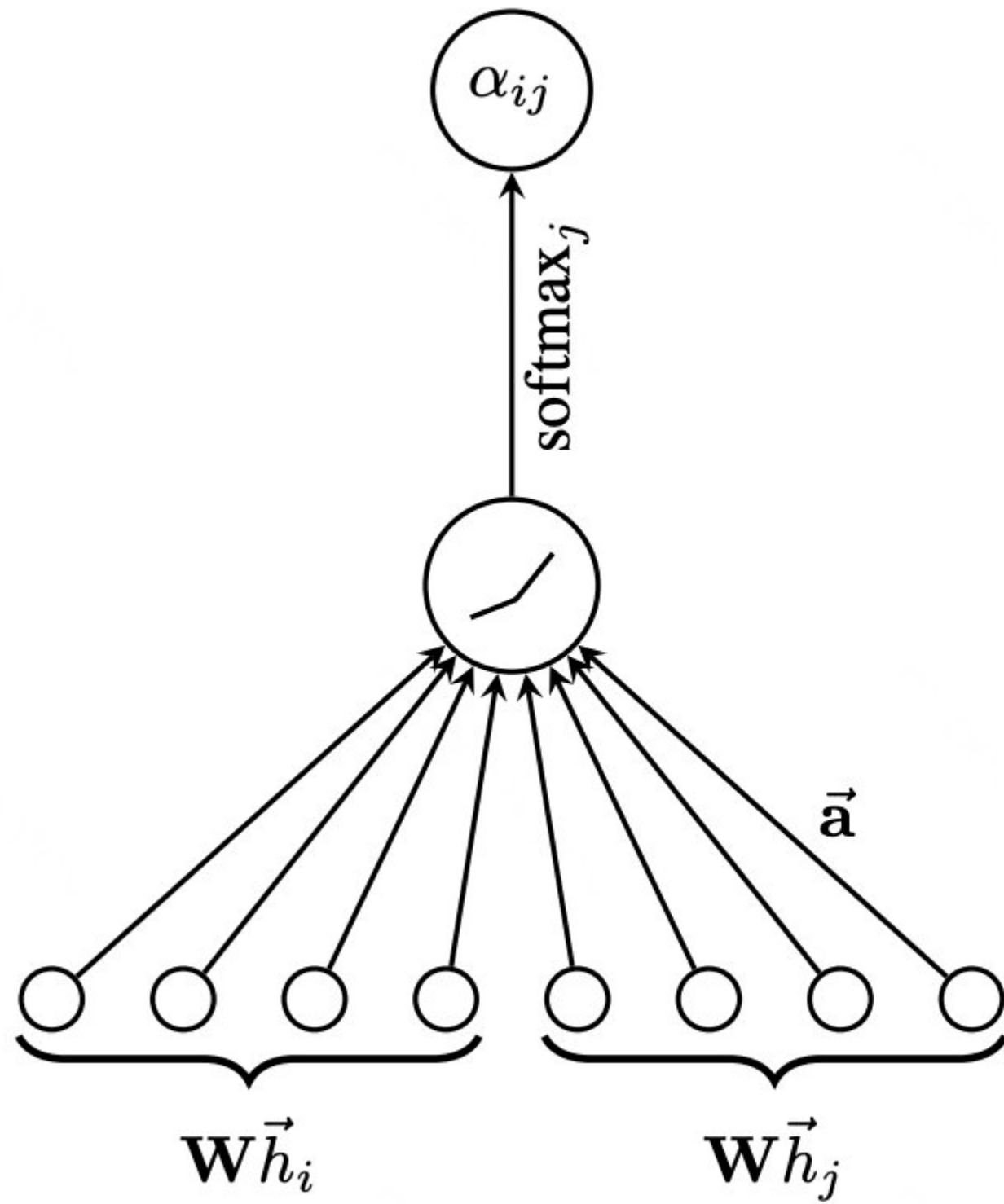
сообщение

$$\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\});$$

$$\mathbf{h}_v^k \leftarrow \sigma \left(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k) \right)$$

обучаемая матрица

GAT



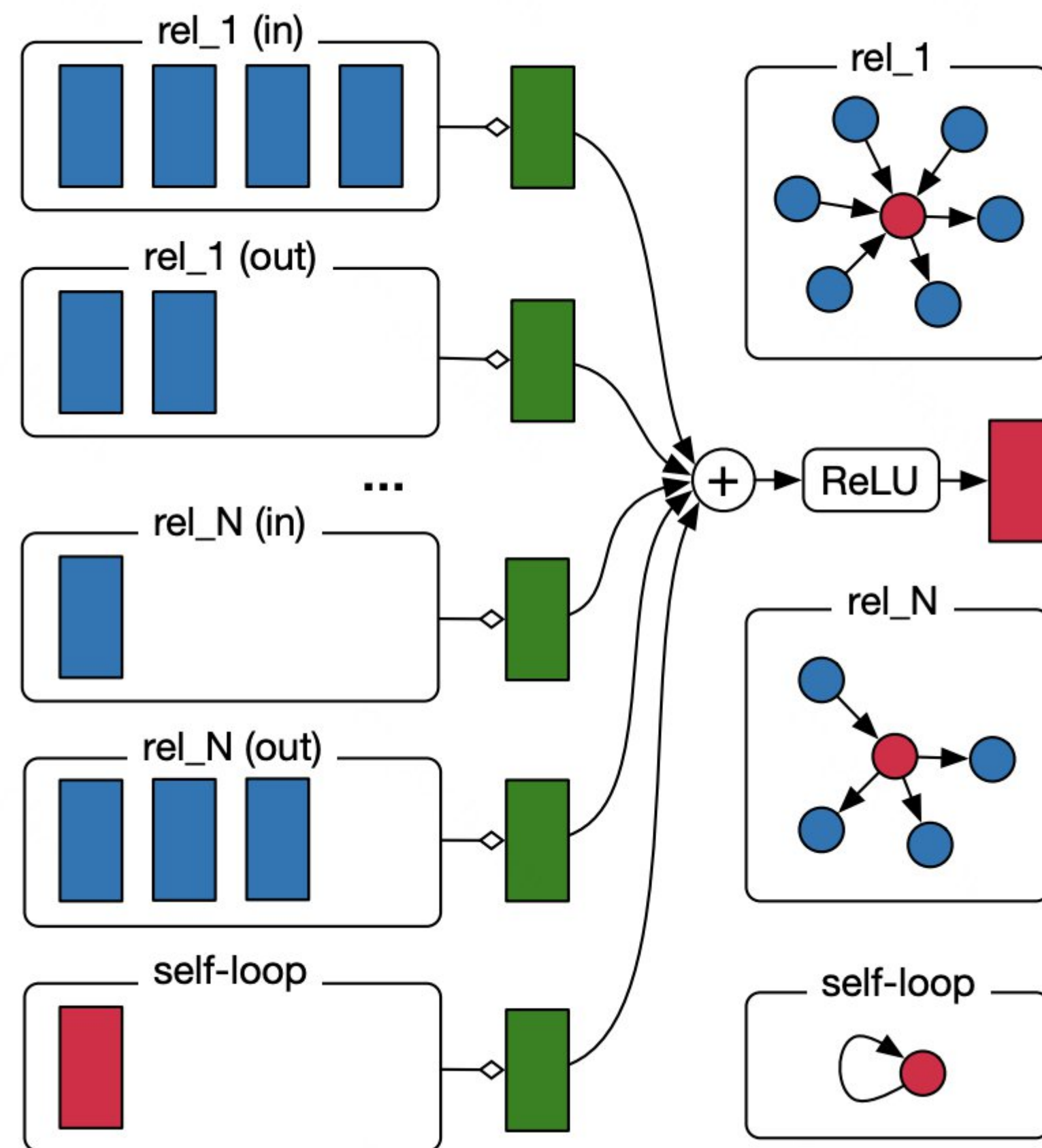
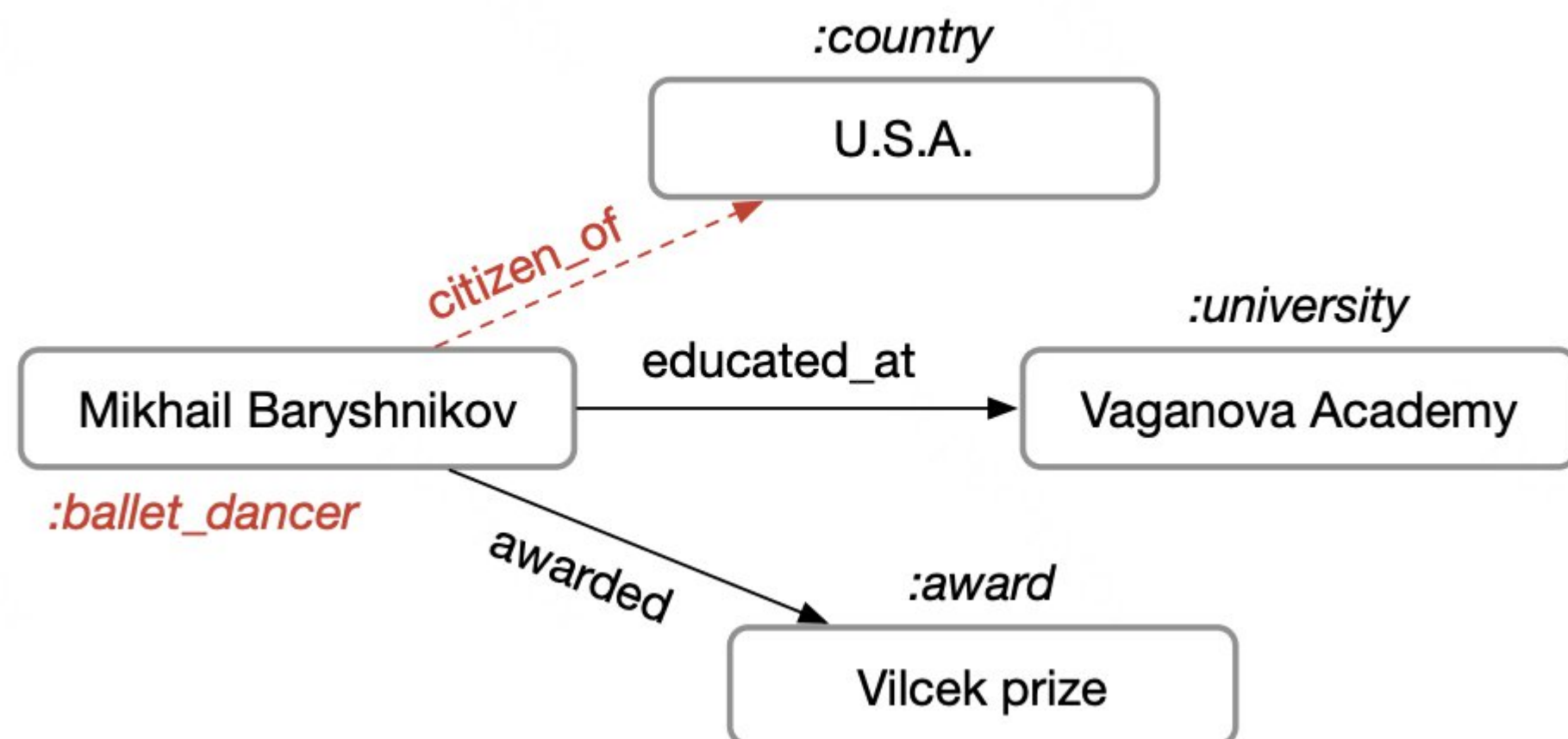
$$e_{ij} = a(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j)$$

$$a : \mathbb{R}^{F'} \times \mathbb{R}^{F'} \rightarrow \mathbb{R}$$

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}.$$

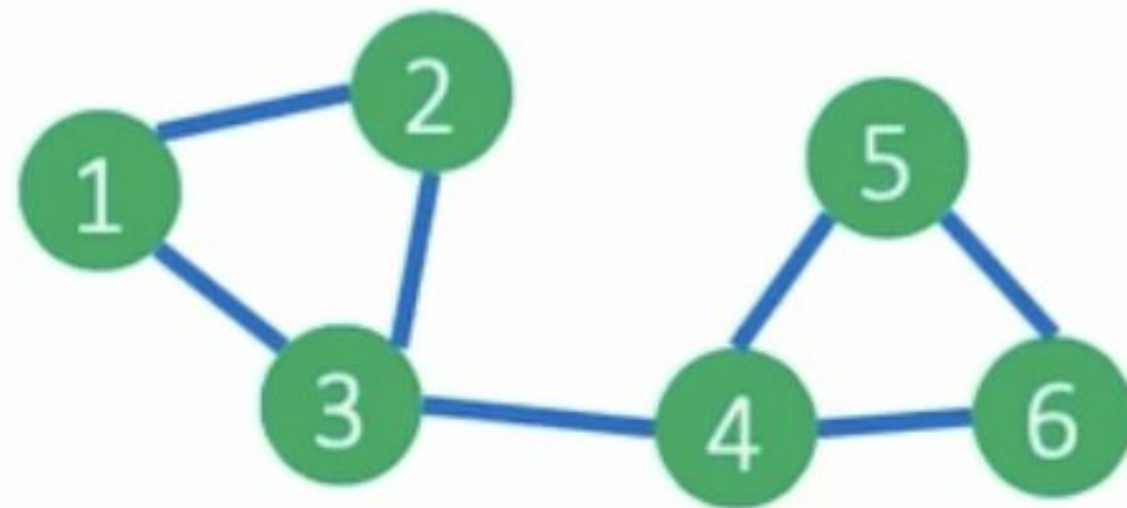
$$\vec{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W}\vec{h}_j \right). \quad \vec{h}'_i = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j \right)$$

RGCN



$$h_i^{(l+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)} \right)$$

Спектральная парадигма(глобальная структура графа)



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \quad D = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{pmatrix}$$

Матрица степеней вершин

Лапласиан

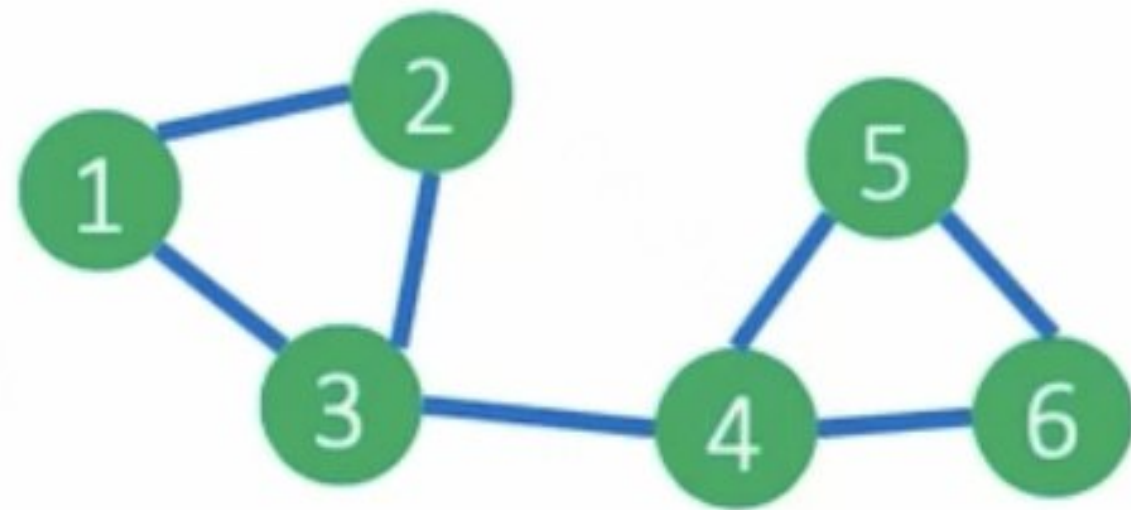
$$L = D - A$$

$$D_{ii} = \sum_j A_{ij}$$

Матрица смежности

$$L = \begin{pmatrix} 2 & -1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{pmatrix}$$

Свойство лаплассиана



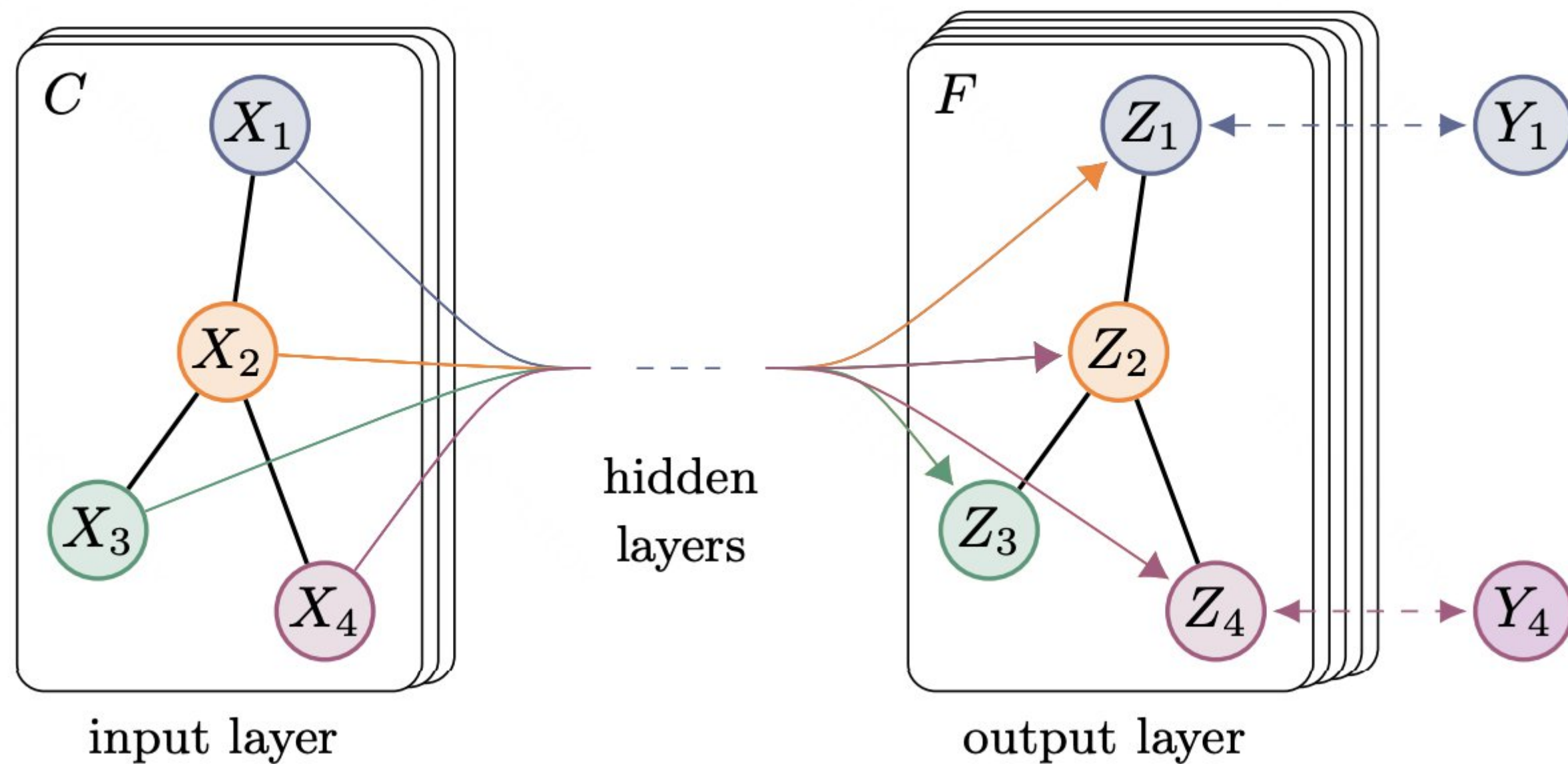
его собственные векторы,
соответствующие положительным
собственным значениям, в порядке
возрастания собственных значений,
описывают разрезы графа

$$x = (-0.46 \quad -0.46 \quad -0.26 \quad 0.26 \quad 0.46 \quad 0.46)$$

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \quad D = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{pmatrix}$$

$$L = \begin{pmatrix} 2 & -1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{pmatrix}$$

GCN

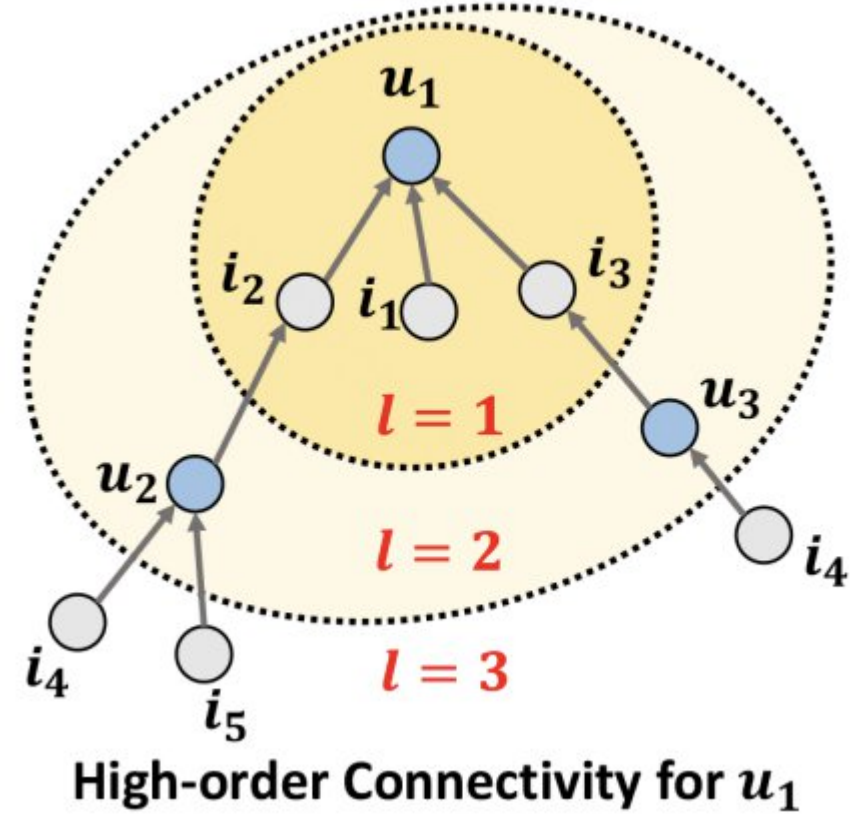
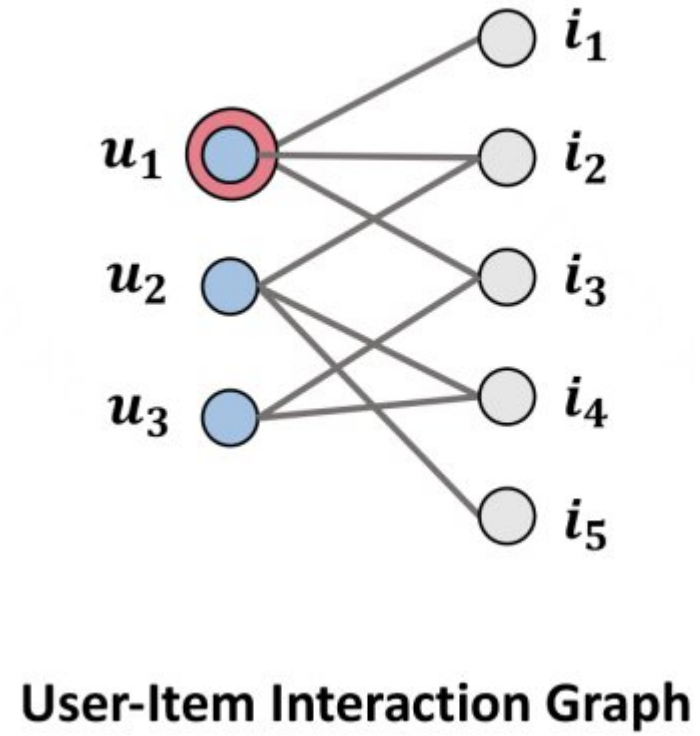


$$h^{t+1} = \theta D^{-1/2} (A + I) D^{-1/2} h^t$$

$$\mathcal{L} = - \sum_{l \in \mathcal{Y}_L} \sum_{f=1}^F Y_{lf} \ln Z_{lf}$$

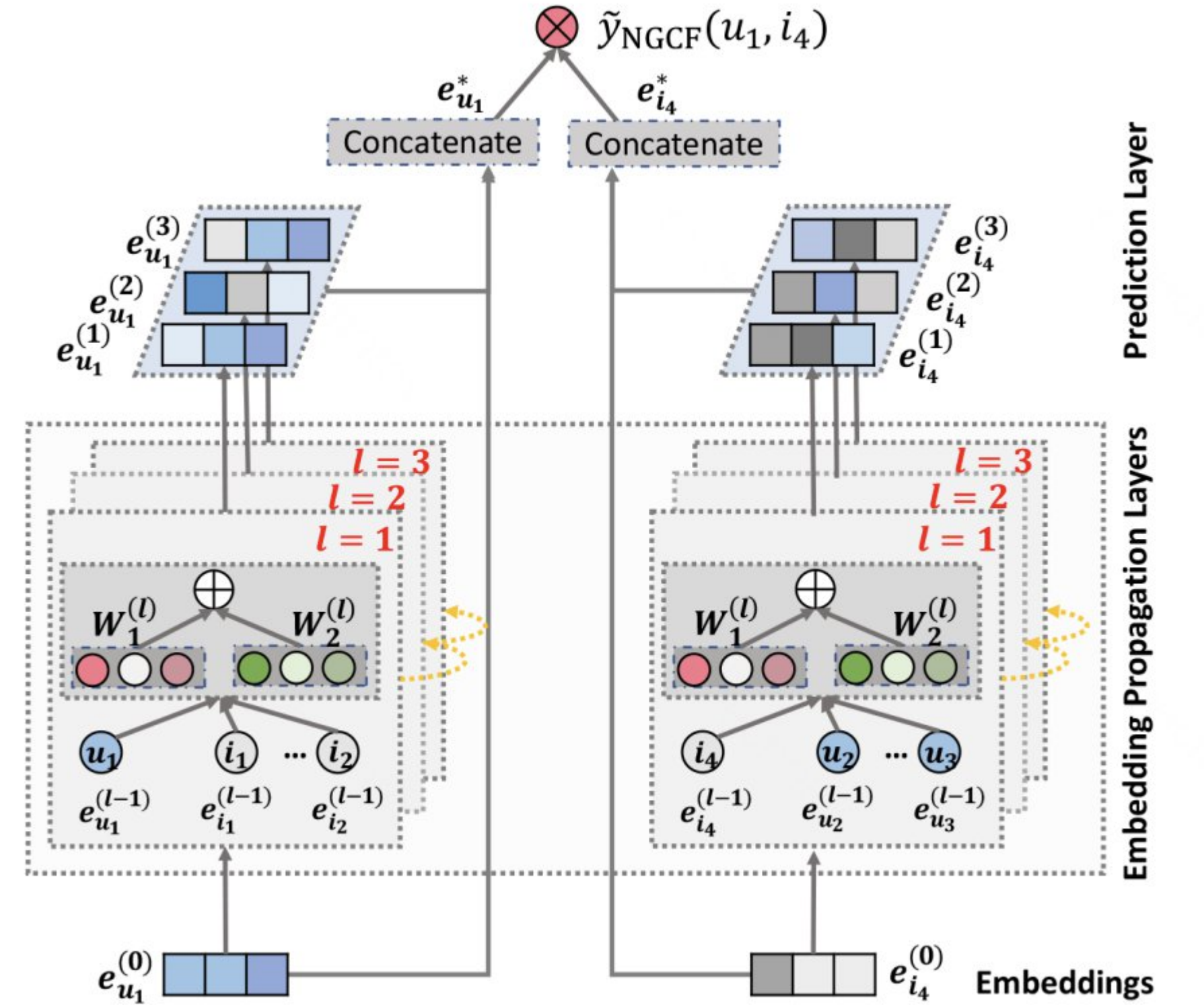
Графовые нейросети

NGCF



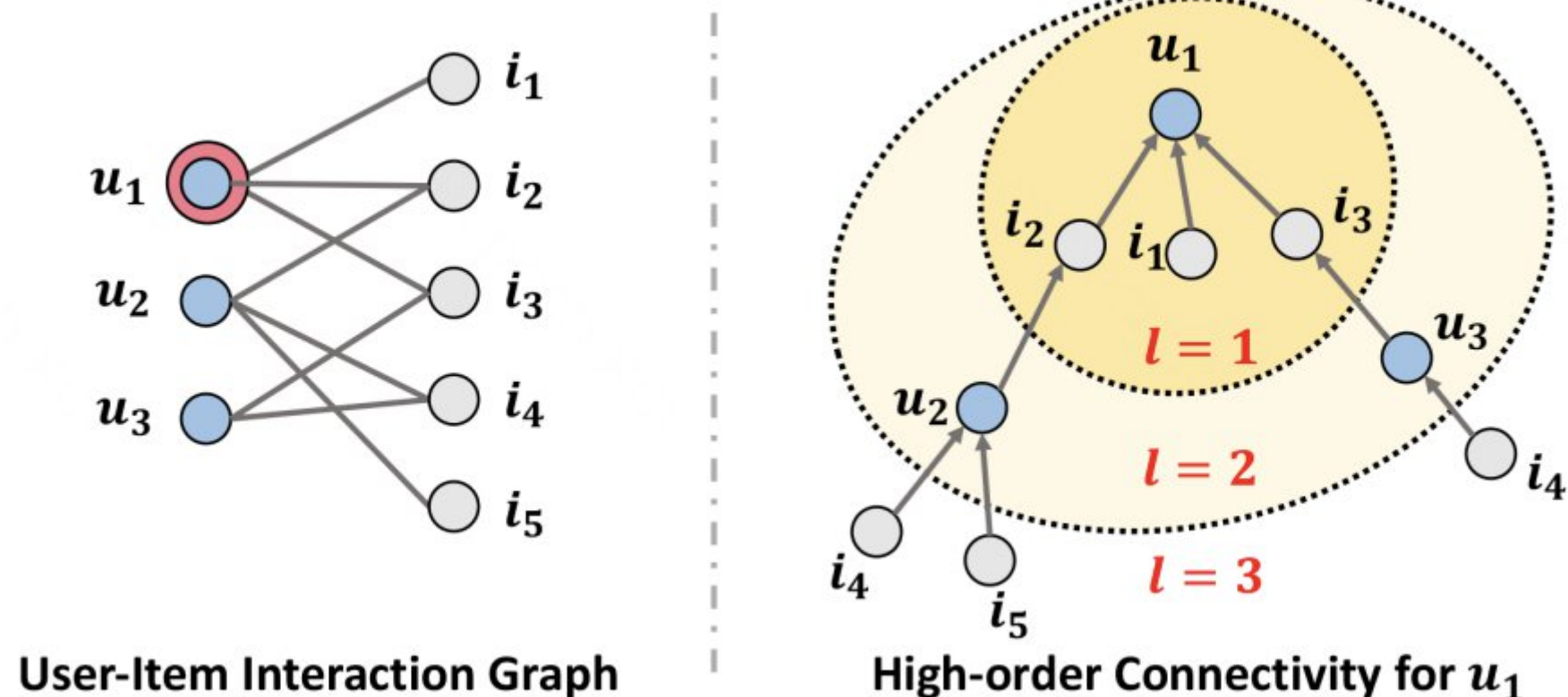
$$\mathbf{m}_{u \leftarrow i} = \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} \left(\mathbf{W}_1 \mathbf{e}_i + \mathbf{W}_2 (\mathbf{e}_i \odot \mathbf{e}_u) \right),$$

$$\mathbf{e}_u^{(1)} = \text{LeakyReLU} \left(\mathbf{m}_{u \leftarrow u} + \sum_{i \in \mathcal{N}_u} \mathbf{m}_{u \leftarrow i} \right),$$



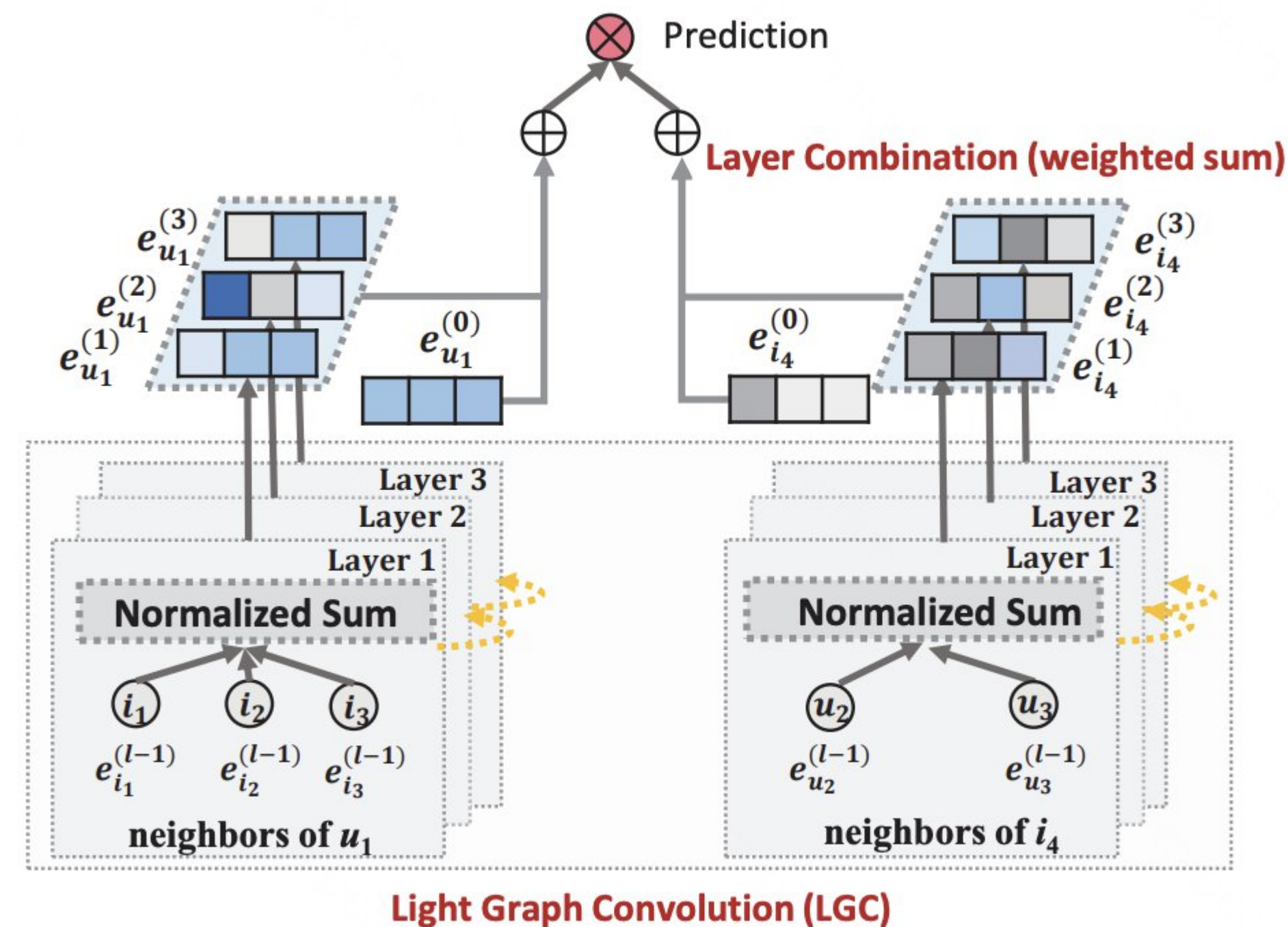
LightGCN

$$L_{BPR} = - \sum_{u=1}^M \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda ||\mathbf{E}^{(0)}||^2$$



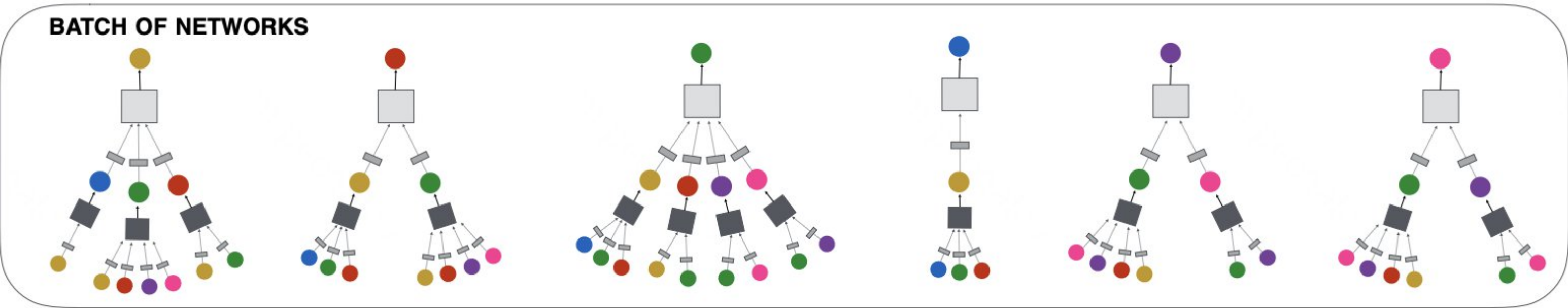
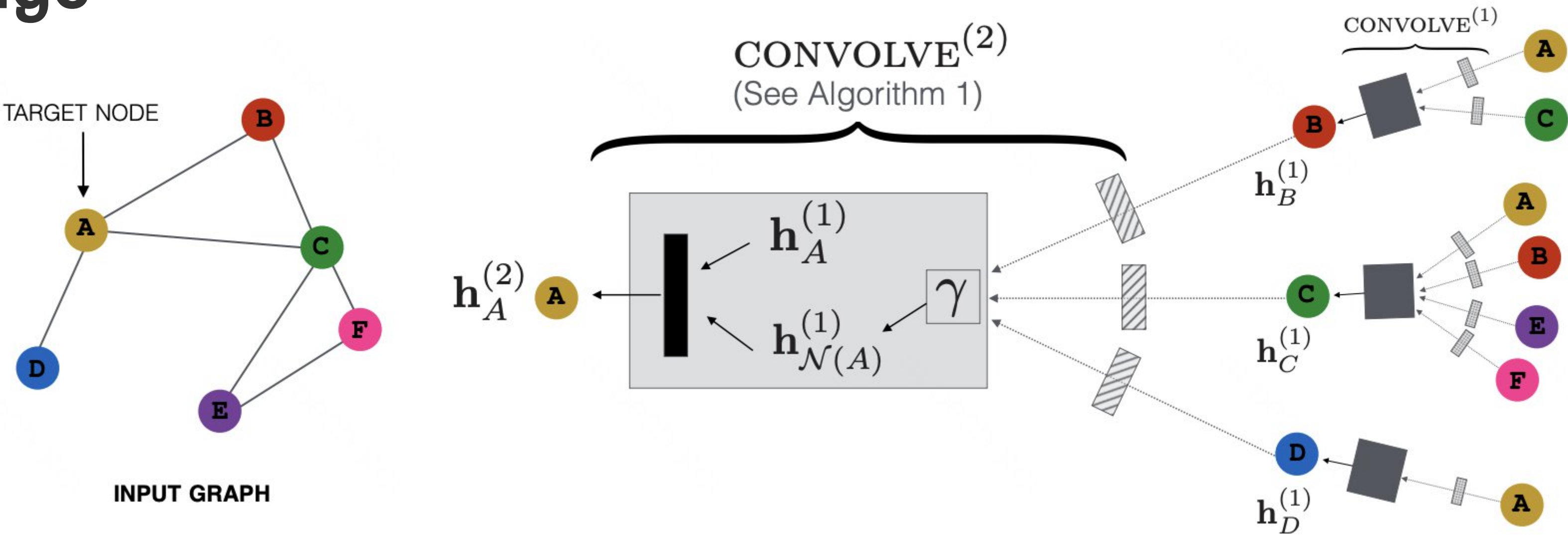
$$\mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|} \sqrt{|\mathcal{N}_i|}} \mathbf{e}_i^{(k)},$$

$$\mathbf{e}_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i|} \sqrt{|\mathcal{N}_u|}} \mathbf{e}_u^{(k)}.$$

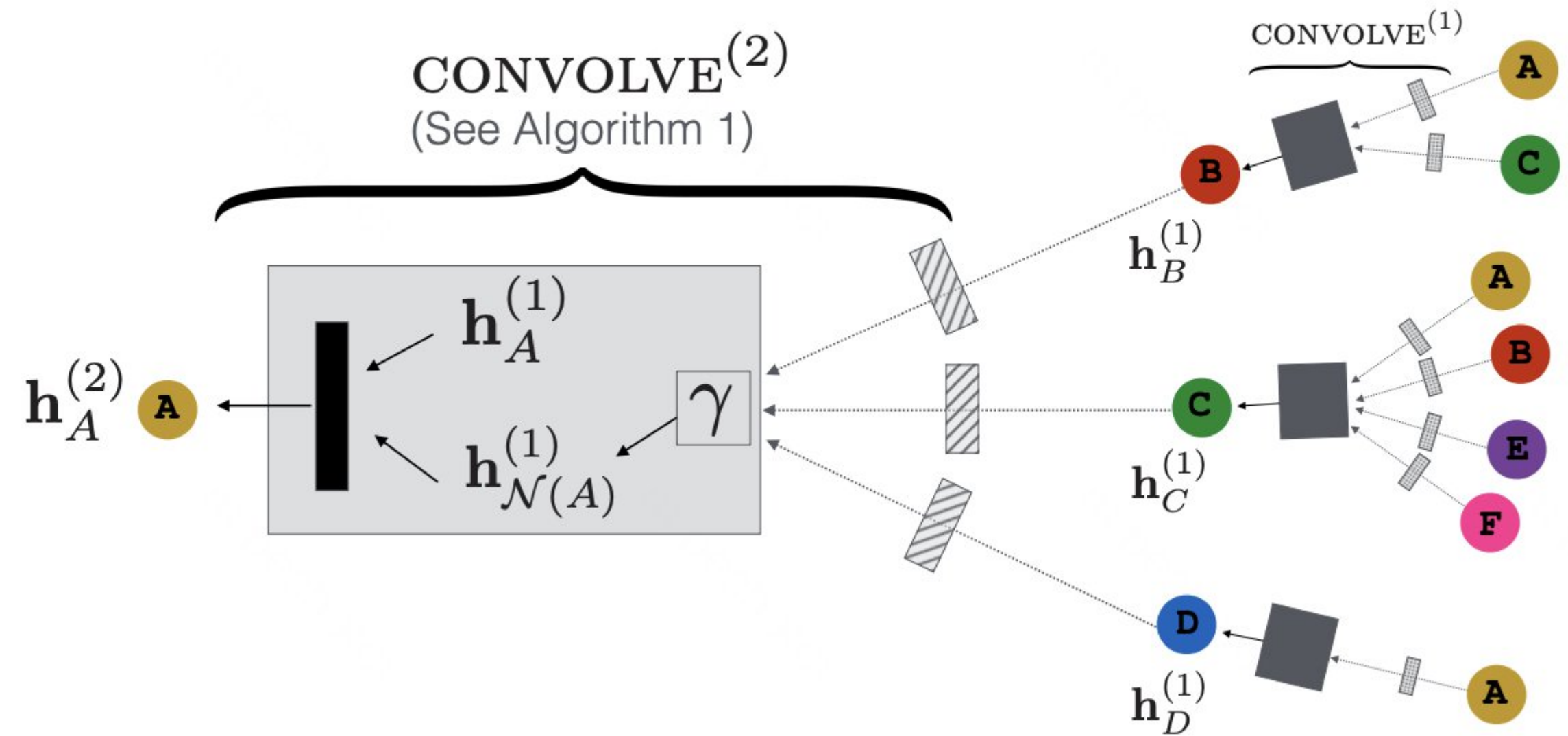
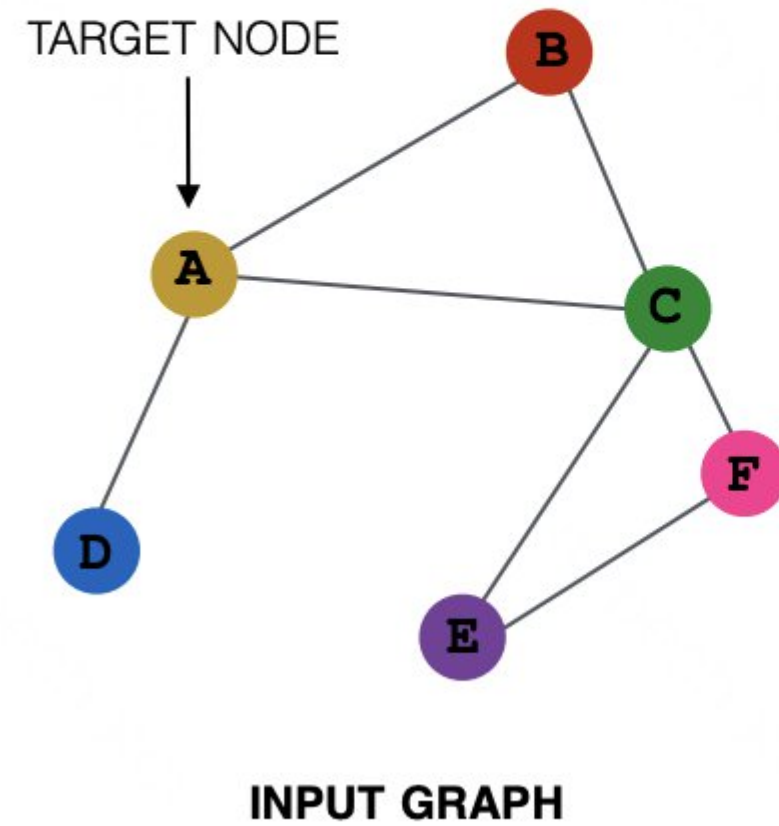


$$\mathbf{e}_u = \sum_{k=0}^K \alpha_k \mathbf{e}_u^{(k)}; \quad \mathbf{e}_i = \sum_{k=0}^K \alpha_k \mathbf{e}_i^{(k)},$$

PinSage

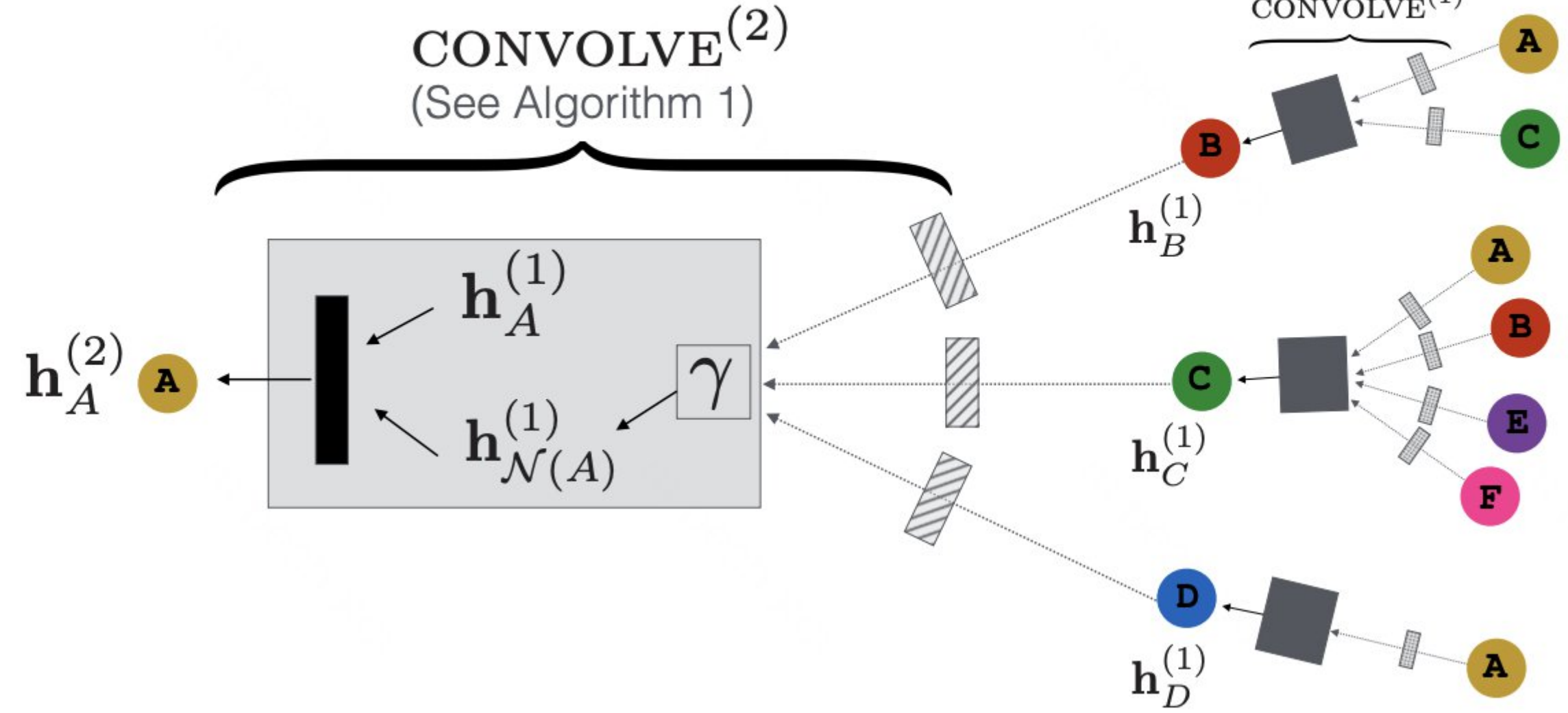
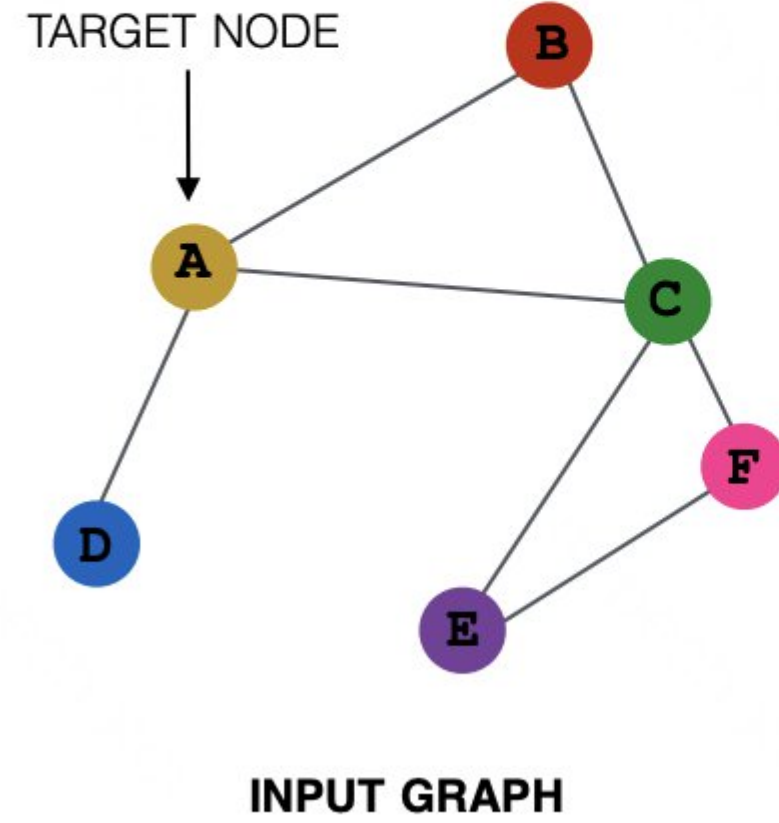


PinSage



- On-the-fly convolutions
- Constructing convolutions via random walks
- Importance pooling:
- Curriculum training

PinSage



Algorithm 1: CONVOLVE

Input : Current embedding $\mathbf{z}_u$ for node u ; set of neighbor embeddings $\{\mathbf{z}_v | v \in \mathcal{N}(u)\}$, set of neighbor weights α ; symmetric vector function $\gamma(\cdot)$

Output: New embedding $\mathbf{z}_u^{\text{NEW}}$ for node u

- 1 $\mathbf{n}_u \leftarrow \gamma(\{\text{ReLU}(\mathbf{Q}\mathbf{h}_v + \mathbf{q}) \mid v \in \mathcal{N}(u)\}, \alpha)$;
 - 2 $\mathbf{z}_u^{\text{NEW}} \leftarrow \text{ReLU}(\mathbf{W} \cdot \text{CONCAT}(\mathbf{z}_u, \mathbf{n}_u) + \mathbf{w})$;
 - 3 $\mathbf{z}_u^{\text{NEW}} \leftarrow \mathbf{z}_u^{\text{NEW}} / \|\mathbf{z}_u^{\text{NEW}}\|_2$
-

Спасибо за внимание